

Assignment Title:

Individual Coursework of Big Data Programming Project

Project Title:

Predicting Bus Fuel Consumption for Operational Efficiency
Using Big Data Analytics and PySpark

Module Code:

ST5011CEM

Date of Submission:

August 4, 2026

Submitted by:

240124

Atom Neupane

Submitted to:

Siddhartha Neupane

Acknowledgment

I would like to sincerely thank Siddhartha Neupane Sir for his guidance, support, and valuable feedback throughout this project. His encouragement helped me successfully complete this work.

I also thank Softwarica College of IT & E-Commerce and Coventry University for providing the opportunity and learning environment for this project.

Finally, I am grateful to the open-source community for tools such as Apache Spark and PySpark, as well as the publicly available datasets that made this project possible.

Table of Contents

Executive Summary	7
Introduction	8
Problem Statement	9
Purpose and Scope	10
Relevance	11
Learning Outcomes Targeted	11
Literature Review / Background	12
Data Collection & Preprocessing	13
Data Sources	13
Tools & Technologies	13
Data Ingestion and Storage	14
Data Cleaning.....	17
Data Preprocessing.....	19
Methodology.....	21
Feature Engineering	22
Model Selection	23
Model Justification.....	25
Linear Regression (Baseline Model)	25
Decision Tree Regression (Non-Linear Model).....	26
Random Forest Regression (Selected Model)	26
Model Comparison.....	27
Data Splitting	28
Algorithm Complexity	29
System Design and Implementation	31
Architecture Overview	31
Software Stack	32
Security Considerations	33
User Interface – Streamlit Dashboard.....	33

Results and Discussion	35
Comparative Model Performance	36
Data Visualization	38
Model Comparison Visualization.....	45
Model Evaluation Visualization.....	48
System Evaluation	52
Limitations of the Approach	53
Future Work	54
Conclusion	55
References	56
Appendix.....	57

Figure 1: Big Data Introduction.....	8
Figure 2: Problem Statement	9
Figure 3: Purpose and Scope.....	10
Figure 4: Data Source	13
Figure 5: Tools & Technologies.....	13
Figure 6: Importing the Bus Fuel Consumption Dataset into PySpark.....	14
Figure 7: Preview of the Bus Fuel Consumption Dataset.....	14
Figure 8: Schema of the Bus Fuel Consumption Dataset.....	15
Figure 9: Dataset Dimensions Showing Total Records and Features	15
Figure 10: Data Ingestion Workflow from BODS to Apache Spark	16
Figure 11: Missing Value Analysis	17
Figure 12: Duplicate Record Analysis Before Cleaning.....	17
Figure 13: Dataset After Removing Duplicate Records	18
Figure 14: Verification of Duplicate Removal.....	18
Figure 15: Encoding of Categorical Features.....	19
Figure 16: Final Feature Vector for Machine Learning.....	20
Figure 17: Overall Project Methodology	21
Figure 18: Feature Engineering Workflow	22
Figure 19: Implementation of the Linear Regression Model	23
Figure 20: Implementation of the Decision Tree Regression Model:	24
Figure 21: Implementation of the Random Forest Regression Model	24
Figure 22: Comparison of Regression Models Using RMSE, MAE, and R^2	27
Figure 23: Splitting the Dataset into Training and Testing Sets.....	28
Figure 24: Relative Training Time Comparison of the Regression Models	29
Figure 25: Prediction Complexity of the Random Forest Regression Model	30
Figure 26: Training Complexity of the Random Forest	30
Figure 27: Overall Architecture of the Bus Fuel Consumption Prediction System	31
Figure 28: Software Stack Used in the Bus Fuel Consumption Prediction System	32
Figure 29: User Interface: 1.....	33
Figure 30: User Interface: 2.....	34
Figure 31: User Interface: 3.....	34
Figure 32: Performance Comparison of Linear Regression, Decision Tree Regression and Random Forest Regression.	36
Figure 33: Feature Importance Generated by the Random Forest Regression Model.....	37
Figure 34: Distribution of Journey Distance.....	38
Figure 35: Distribution of Fuel Consumption.....	39
Figure 36: Average Fuel Consumption by Departure hour	40
Figure 37: Fuel Consumption by Traffic Level	41

Figure 38: Fuel Consumption by Weather Condition.....	42
Figure 39: Distance vs Fuel Consumption	43
Figure 40: Correlation Heatmap of Numerical Features	44
Figure 41: R^2 Score Comparison.....	45
Figure 42: MAE Comparison	46
Figure 43: RMSE Comparison.....	47
Figure 44: Actual vs Predicted Fuel Consumption Using the Random Forest Model.	48
Figure 45: Residual Error Distribution of the Random Forest Prediction Model.....	49
Figure 46: Prediction Error Analysis of the Random Forest Model.	50
Figure 47: Q-Q Plot of Residual Errors for the Random Forest Regression Model.	51

Executive Summary

The project uses a dataset containing approximately 114,000 bus operation records. The dataset includes information such as bus operators, route details, travel distance, number of stops, departure hour, traffic conditions, weather, average speed, delays, and fuel consumption. Since the dataset is relatively large, Apache Spark was selected to process the data efficiently and perform scalable machine learning tasks.

The project followed a structured workflow consisting of data collection, data cleaning, preprocessing, exploratory data analysis, feature engineering, model development, and performance evaluation. Missing values, duplicate records, and inconsistent data were handled during preprocessing to improve data quality. Several visualizations were also created to better understand the relationships between different operational variables and fuel consumption.

Three regression algorithms Linear Regression, Decision Tree Regression, and Random Forest Regression were implemented using the PySpark Machine Learning library. Their performance was evaluated using Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and the coefficient of determination (R^2). The results showed that the Random Forest Regression model produced the highest prediction accuracy among the evaluated models, making it the most suitable model for this dataset.

Overall, the project demonstrates how big data technologies and machine learning can be combined to analyze transportation data and predict fuel consumption effectively. The developed solution provides practical insights that could support transport operators in reducing fuel usage, lowering operational costs, and improving overall fleet efficiency. It also highlights the importance of data-driven decision-making in modern transportation systems and the growing role of big data analytics in solving real-world operational challenges.

Introduction

This project predicts bus fuel consumption using historical operational data to improve transport efficiency and support better decision-making. It addresses the challenge of varying fuel usage caused by factors such as traffic conditions, weather, travel distance, and delays. Using Apache Spark and PySpark, the project covers data ingestion, preprocessing, feature engineering, and machine learning model evaluation. A Random Forest Regression model was developed to provide accurate fuel consumption predictions, helping improve operational planning and resource management.



Figure 1: Big Data Introduction

Problem Statement

Fuel consumption is one of the major operational expenses in public transportation. Factors such as traffic congestion, weather conditions, travel distance, delays, and driving patterns can cause significant variations in fuel usage. Traditional methods of estimating fuel consumption often fail to capture these changing conditions, resulting in inefficient resource planning and increased operational costs.

With the availability of large-scale transportation data, there is an opportunity to use big data analytics and machine learning to make more accurate fuel consumption predictions. This project addresses this challenge by using Apache Spark and PySpark to analyse bus operational data and develop predictive models. The goal is to improve prediction accuracy, support better operational planning, and help transport operators optimize fuel usage and reduce costs.

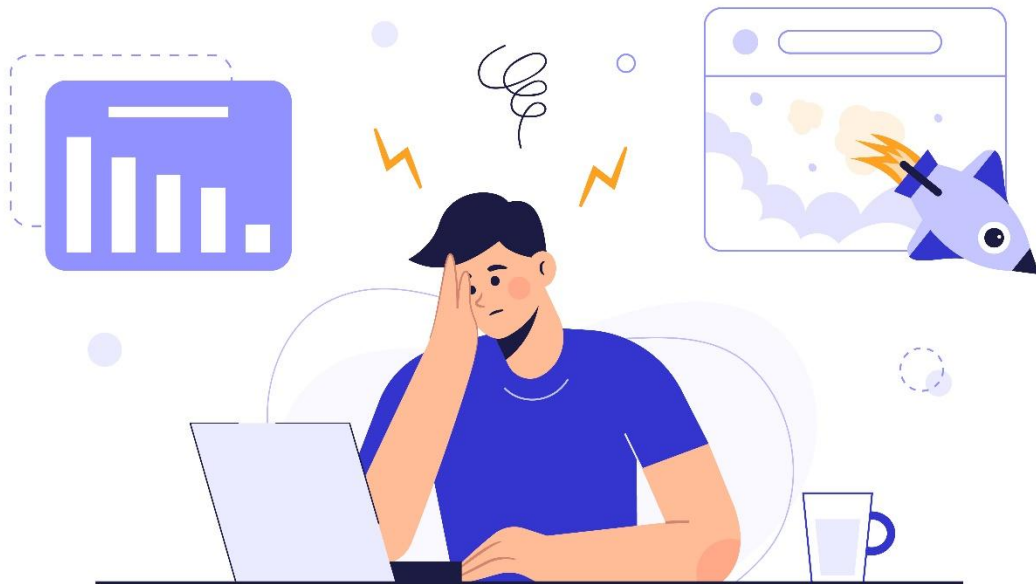


Figure 2: Problem Statement

Purpose and Scope

The purpose of this project is to predict bus fuel consumption using machine learning and big data technologies. By analyzing operational data such as travel distance, traffic conditions, weather, average speed, delays, and the number of stops, the project aims to identify the factors that affect fuel usage and build an accurate prediction model.

The project uses Apache Spark and PySpark to process a large dataset efficiently. It includes data collection, data cleaning, feature engineering, model development, and performance evaluation. Three machine learning models Linear Regression, Decision Tree Regression, and Random Forest Regression are compared to determine the most accurate model for predicting fuel consumption. The findings of this project can help transport operators improve fuel efficiency, reduce operational costs, and support better decision-making.

Bus Fuel Consumption Prediction Project

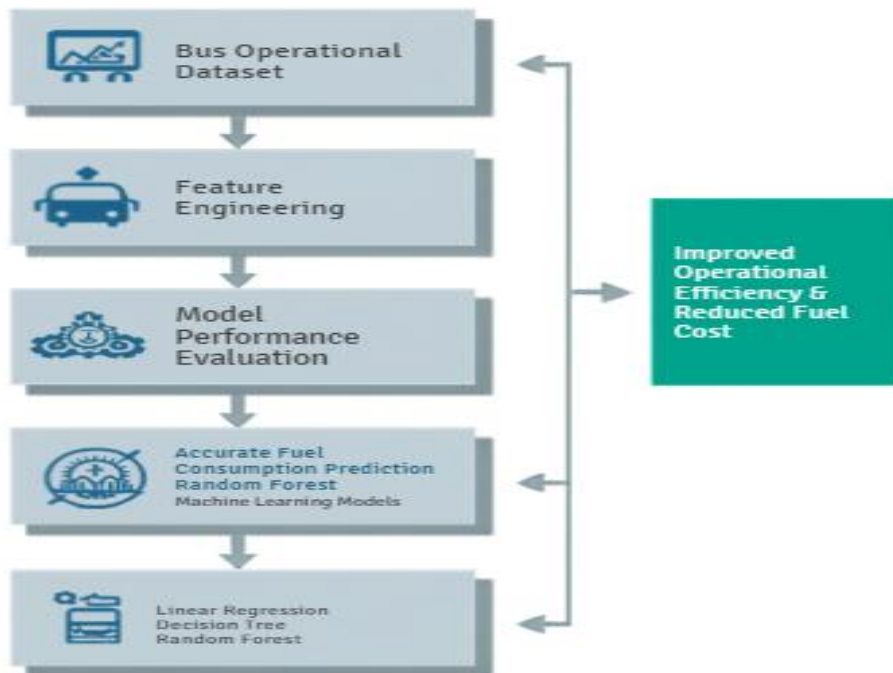


Figure 3: Purpose and Scope

Relevance

With the increasing use of big data and machine learning, the transportation industry is finding new ways to improve its services and reduce operational costs. Predicting bus fuel consumption helps transport operators understand how different factors, such as traffic, weather, travel distance, and delays, affect fuel usage. This information can support better planning, improve fuel efficiency, and reduce unnecessary expenses.

This project is relevant because it demonstrates how Apache Spark and PySpark can be used to analyze large transportation datasets and develop accurate prediction models. It also highlights the value of data-driven decision-making in improving public transportation and promoting more efficient and sustainable operations.

Learning Outcomes Targeted

B1	Data Processing	Processed and cleaned the bus fuel consumption dataset using PySpark.
B2	Big Data Technologies	Used Apache Spark and PySpark for large-scale data analysis.
B3	Machine Learning	Built Linear Regression, Decision Tree, and Random Forest models.
B4	Feature Engineering	Prepared and selected features to improve prediction accuracy.
B5	Model Evaluation	Compared models using RMSE, MAE, and R ² Score.
B6	Data Visualization	Created charts to analyze fuel consumption patterns.
B7	Problem Solving	Applied predictive analytics to improve operational efficiency.
B8	Documentation	Documented the project and presented the findings clearly.

Literature Review / Background

Fuel consumption is one of the most important factors affecting the operational cost of public transportation. Transport operators continuously seek ways to reduce fuel usage while maintaining reliable services. Predicting fuel consumption using historical operational data has become an effective approach for improving fleet management, reducing unnecessary fuel costs, and supporting sustainable transportation.

Recent studies have shown that machine learning techniques can analyze operational factors such as travel distance, average speed, traffic conditions, weather, delays, and the number of stops to predict fuel consumption with good accuracy. These approaches help transport organizations identify the key factors influencing fuel usage and make better operational decisions.

The growing availability of transportation data has also increased the use of big data technologies. Apache Spark and its Python API, PySpark, enable large datasets to be processed efficiently through distributed computing. Their scalability and built-in machine learning libraries make them suitable for developing predictive models using large transportation datasets.

In this project, a dataset containing approximately 114,000 bus operational records is analyzed using Apache Spark and PySpark. The data is cleaned, preprocessed, and transformed before applying Linear Regression, Decision Tree Regression, and Random Forest Regression models. The performance of these models is compared using evaluation metrics such as RMSE, MAE, and R^2 Score to identify the most accurate approach for predicting bus fuel consumption. The findings demonstrate how big data analytics and machine learning can support operational efficiency and reduce fuel-related costs in public transportation.

Data Collection & Preprocessing

Data Sources

The primary data source for this project is the UK Bus Open Data Service (BODS), which provides publicly available bus operational data in the TransXChange XML format. The XML data was converted into a structured CSV dataset containing approximately 114,000 records, including information such as operator, travel distance, number of stops, departure hour, traffic level, weather conditions, average speed, delay minutes, and fuel consumption. The final CSV dataset was then imported into Apache Spark using PySpark for data preprocessing, analysis, and machine learning model development.



Figure 4: Data Source

Tools & Technologies

This project was developed using several tools and technologies to support data processing, machine learning, and visualization. Python was used as the primary programming language because of its simplicity and extensive data science libraries. Apache Spark and PySpark were used to process the large bus fuel consumption dataset efficiently through distributed computing. Jupyter Notebook provided an interactive environment for coding, testing, and analyzing the data, while Matplotlib was used to create charts for exploratory data analysis and model evaluation.

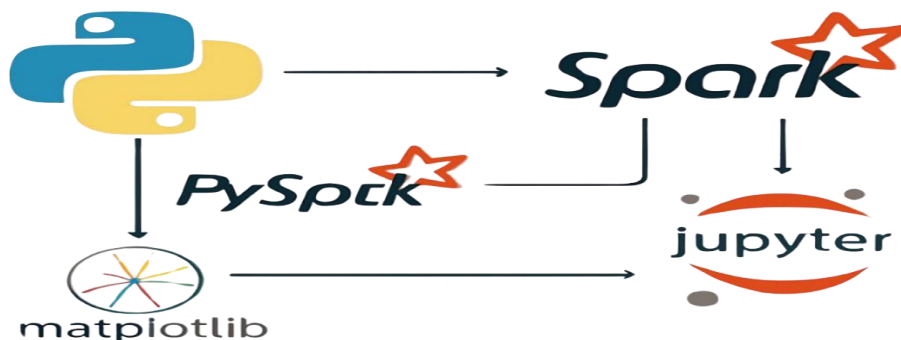


Figure 5: Tools & Technologies

Data Ingestion and Storage

The bus fuel consumption dataset was stored in CSV format after converting the original XML data obtained from the UK Bus Open Data Service (BODS). The dataset was imported into Apache Spark using PySpark with schema inference enabled, allowing the data to be loaded efficiently. Spark Data Frames were then used throughout the project for preprocessing, feature engineering, model training, and evaluation.

```
[2]: df = spark.read.csv(
    r"C:\Users\ASUS\Documents\Bus project\data\bus_fuel_consumption_114k.csv",
    header=True,
    inferSchema=True
)

df.show(5)
```

Operator	Service_Code	Line_Name	Distance	Number_of_Stops	Departure_Hour	Peak_Hour	Day_Type	Weather	Traffic_Level	Delay_Minutes	Average_Speed	Fuel_Consumption	Efficiency_Class
Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Rain	High	17	40	210.65	Low
Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Rain	High	13	39	209.38	Medium
Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Clear	High	21	36	195.43	Low
Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Clear	Medium	5	53	168.42	Medium
Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Rain	High	27	25	209.19	Low

Figure 6: Importing the Bus Fuel Consumption Dataset into PySpark

```
1]: df.show(10, truncate=False)
```

Operator	Service_Code	Line_Name	Distance	Number_of_Stops	Departure_Hour	Peak_Hour	Day_Type	Weather	Traffic_Level	Delay_Minutes	Average_Speed	Fuel_Consumption	Efficiency_Class
Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Rain	High	17	40	210.65	Low
Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Rain	High	13	39	209.38	Medium
Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Clear	High	21	36	195.43	Low
Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Clear	Medium	5	53	168.42	Medium
Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Rain	High	27	25	209.19	Low
Stagecoach South	PK0002571:21	46	437	264	7	1	Weekend	Clear	Medium	5	52	64.9	Medium
Stagecoach South	PK0002571:21	46	437	264	7	1	Weekend	Clear	High	15	43	97.29	Medium
Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Rain	High	14	43	11.2	Medium
Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Clear	High	12	43	93.19	Medium
Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Rain	High	18	39	10.75	Low

only showing top 10 rows

Figure 7: Preview of the Bus Fuel Consumption Dataset

```
df.printSchema()

print("Rows :", df.count())

print("Columns :", len(df.columns))

root
 |-- Operator: string (nullable = true)
 |-- Service_Code: string (nullable = true)
 |-- Line_Name: string (nullable = true)
 |-- Distance: integer (nullable = true)
 |-- Number_of_Stops: integer (nullable = true)
 |-- Departure_Hour: integer (nullable = true)
 |-- Peak_Hour: integer (nullable = true)
 |-- Day_Type: string (nullable = true)
 |-- Weather: string (nullable = true)
 |-- Traffic_Level: string (nullable = true)
 |-- Delay_Minutes: integer (nullable = true)
 |-- Average_Speed: integer (nullable = true)
 |-- Fuel_Consumption: double (nullable = true)
 |-- Efficiency_Class: string (nullable = true)

Rows : 114000
Columns : 14
```

Figure 8: Schema of the Bus Fuel Consumption Dataset

```
df.describe().show()
```

summary	Operator	Service_Code	Line_Name	Distance	Number_of_Stops	Departure_Hour	Peak_Hour	Day_Type	Weather
count	114000	114000	114000	114000	114000	114000	114000	114000	114000
mean	114000	114000	114000	114000	114000	114000	114000	114000	114000
stddev	114000	114000	114000	114000	114000	114000	114000	114000	114000
min	114000	114000	114000	114000	114000	114000	114000	114000	114000
High	114000	114000	114000	114000	114000	114000	114000	114000	114000
max	114000	114000	114000	114000	114000	114000	114000	114000	114000
Medium	114000	114000	114000	114000	114000	114000	114000	114000	114000

Figure 9: Dataset Dimensions Showing Total Records and Features

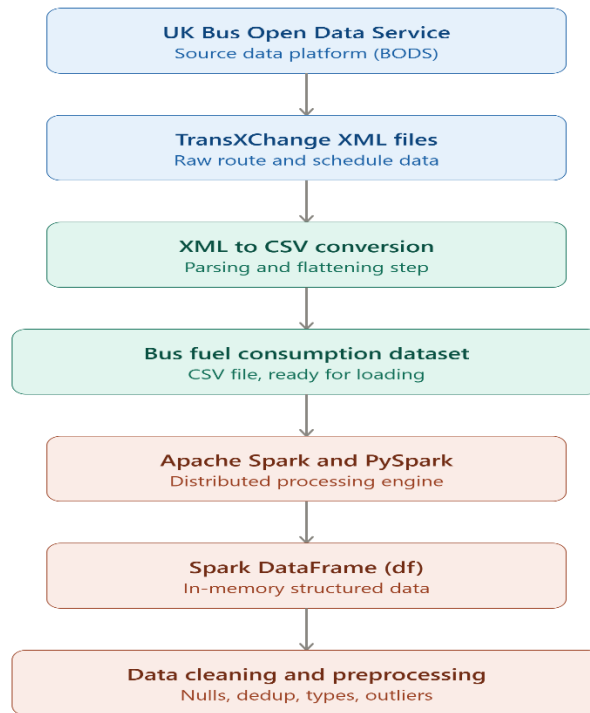


Figure 10: Data Ingestion Workflow from BODS to Apache Spark

Data Cleaning

Data cleaning was performed to improve the quality and reliability of the dataset before building the machine learning models. After loading the data into PySpark, the dataset was examined for missing values, duplicate records, and incorrect data types. Any inconsistencies found during this stage were corrected to ensure the dataset was suitable for further analysis.

The cleaning process included checking the dataset structure, handling missing values, removing duplicate records, and verifying that each column had the correct data type. These steps helped produce a clean and consistent dataset for feature engineering and model training.

```
from pyspark.sql.functions import col, when, count

missing = df.select([
    count.when(col(c).isNull(), c).alias(c)
    for c in df.columns
])

missing.show()
```

Operator	Service_Code	Line_Name	Distance	Number_of_Stops	Departure_Hour	Peak_Hour	Day_Type	Weather	Traffic_Level	Delay_Minutes	Average_Speed	Fuel_Consumption	Efficiency_Class
0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figure 11: Missing Value Analysis

```
[8]: total_rows = df.count()
     unique_rows = df.dropDuplicates().count()

     print("Total Rows:", total_rows)
     print("Unique Rows:", unique_rows)
     print("Duplicate Rows:", total_rows - unique_rows)
```

Total Rows: 114000
Unique Rows: 113936
Duplicate Rows: 64

Figure 12: Duplicate Record Analysis Before Cleaning

```
df = df.dropDuplicates()

print("Rows after removing duplicates:", df.count())
```

Rows after removing duplicates: 113936

Figure 13: Dataset After Removing Duplicate Records

```
[1]: total_rows = df.count()
      unique_rows = df.dropDuplicates().count()

      print("Total Rows:", total_rows)
      print("Unique Rows:", unique_rows)
      print("Duplicate Rows:", total_rows - unique_rows)
```

Total Rows: 113936
Unique Rows: 113936
Duplicate Rows: 0

Figure 14: Verification of Duplicate Removal

Data Preprocessing

Data preprocessing is an essential step in the machine learning workflow because it transforms raw data into a structured format suitable for analysis and model training. After cleaning the dataset, several preprocessing techniques were applied using PySpark to improve data quality and prepare the features for predictive modelling.

The following preprocessing tasks were performed:

- Selected the relevant features required for fuel consumption prediction.
- Encoded categorical variables into numerical values using PySpark indexing techniques.
- Combined all input features into a single feature vector using VectorAssembler.
- Split the dataset into training (80%) and testing (20%) sets for model development and evaluation.
- Prepared the processed dataset for training machine learning models, including Linear Regression, Decision Tree Regression, and Random Forest Regression.

```
3]: df.select(
    "Operator",
    "Operator_Index",
    "Weather",
    "Weather_Index",
    "Traffic_Level",
    "Traffic_Index",
    "Day_Type",
    "Day_Index"
).show(10, truncate=False)
```

Operator	Operator_Index	Weather	Weather_Index	Traffic_Level	Traffic_Index	Day_Type	Day_Index
Stagecoach South	0.0	Rain	1.0	High	0.0	Weekday	0.0
Stagecoach South	0.0	Fog	2.0	Low	2.0	Weekday	0.0
Stagecoach South	0.0	Clear	0.0	High	0.0	Weekday	0.0
Stagecoach South	0.0	Clear	0.0	Medium	1.0	Weekday	0.0
Stagecoach South	0.0	Rain	1.0	Medium	1.0	Weekday	0.0
Stagecoach South	0.0	Clear	0.0	Medium	1.0	Weekday	0.0
Stagecoach South	0.0	Fog	2.0	Low	2.0	Weekday	0.0
Stagecoach South	0.0	Clear	0.0	Medium	1.0	Weekend	1.0
Stagecoach South	0.0	Clear	0.0	Medium	1.0	Weekday	0.0
Stagecoach South	0.0	Clear	0.0	High	0.0	Weekend	1.0

only showing top 10 rows

Figure 15: Encoding of Categorical Features

```
25]: from pyspark.ml.feature import VectorAssembler
```

```
assembler = VectorAssembler(  
    inputCols=feature_columns,  
    outputCol="features"  
)
```

```
df = assembler.transform(df)
```

```
26]: df.select(  
    "features",  
    "Fuel_Consumption"  
)<div data-bbox="158 332 753 473" data-label="Text">

| features                                         | Fuel_Consumption |
|--------------------------------------------------|------------------|
| [437.0,264.0,7.0,1.0,19.0,35.0,0.0,1.0,0.0,0.0]  | 209.81           |
| [437.0,109.0,7.0,1.0,8.0,50.0,0.0,2.0,2.0,0.0]   | 158.65           |
| [721.0,3854.0,7.0,1.0,16.0,40.0,0.0,0.0,0.0,0.0] | 393.67           |
| [721.0,3854.0,7.0,1.0,7.0,46.0,0.0,0.0,1.0,0.0]  | 335.09           |
| [721.0,3854.0,7.0,1.0,12.0,46.0,0.0,1.0,1.0,0.0] | 359.41           |


```

```
only showing top 5 rows
```

Figure 16: Final Feature Vector for Machine Learning

Methodology

This chapter describes the methodology followed to develop the bus fuel consumption prediction system. The project follows a structured machine learning workflow, beginning with data collection and preprocessing, followed by feature engineering, model training, evaluation, and prediction. Apache Spark and PySpark were used throughout the process to efficiently handle the large dataset and implement the machine learning models.

The overall methodology ensures that the dataset is prepared correctly before training the models and evaluating their performance. Three regression algorithms were implemented and compared to identify the most accurate model for predicting bus fuel consumption.

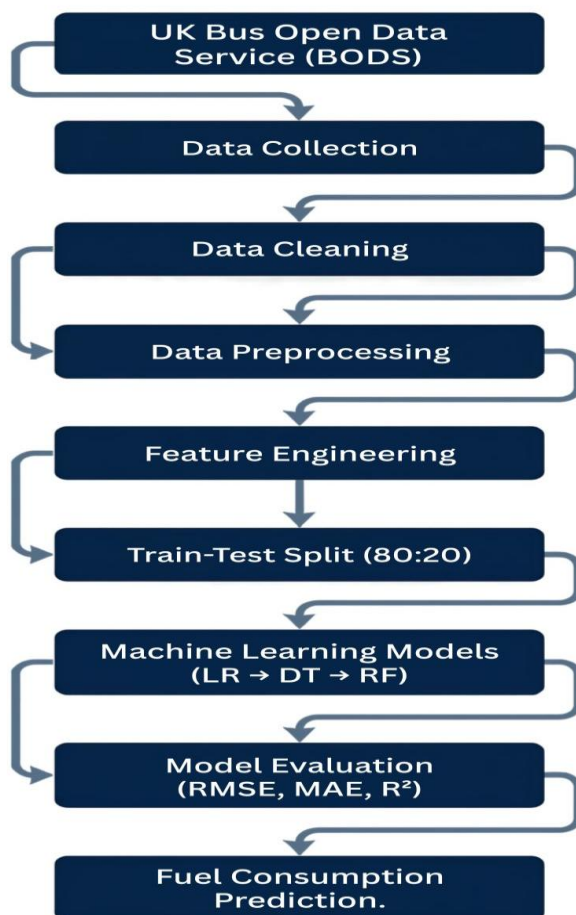


Figure 17: Overall Project Methodology

Feature Engineering

Feature engineering was an important step in preparing the dataset for machine learning. At this stage, the most relevant variables related to bus fuel consumption were identified and organized to help the models learn more effectively. Only the features that could influence fuel usage were selected, while unnecessary information was excluded from the analysis.

The main feature engineering tasks included:

- Selecting the operational features that have the greatest impact on fuel consumption.
- Using variables such as travel distance, average speed, number of stops, departure hour, weather conditions, traffic level, and delay minutes as input features.
- Removing columns that were not required for prediction.
- Organizing the selected features into a format suitable for machine learning models.
- Preparing the final dataset for model training and evaluation.

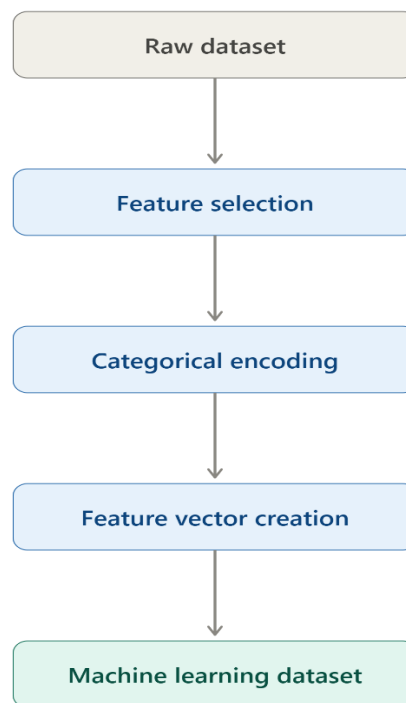


Figure 18: Feature Engineering Workflow

Model Selection

Selecting the right machine learning model is an important step in building an accurate prediction system. In this project, three different regression models were chosen to predict bus fuel consumption and compare their performance. Each model has its own strengths and was selected to understand how different algorithms perform on the same dataset.

The models used in this project are:

- Linear Regression – Used as a simple baseline model to understand the relationship between the input features and fuel consumption.
- Decision Tree Regression – Selected because it can capture more complex and non-linear relationships within the data.
- Random Forest Regression – Chosen for its ability to improve prediction accuracy by combining multiple decision trees and reducing overfitting.

```
[28]: from pyspark.ml.regression import LinearRegression

lr = LinearRegression(
    featuresCol="features",
    labelCol="Fuel_Consumption"
)

lr_model = lr.fit(train_data)
```

```
[29]: lr_predictions = lr_model.transform(test_data)

lr_predictions.select(
    "Fuel_Consumption",
    "prediction"
).show(10, truncate=False)
```

```
+-----+-----+
|Fuel_Consumption|prediction|
+-----+-----+
|147.03          |158.02065001937353|
|145.52          |158.26063589586704|
|148.5           |158.97994636410505|
|152.39          |159.73914661821823|
|148.08          |159.85946313708624|
|148.3           |161.69762909355646|
|113.2           |46.68365124531062 |
|124.73          |102.21253638177899|
|159.98          |182.42135092751744|
|153.87          |183.42053705812415|
+-----+-----+
only showing top 10 rows
```

Figure 19: Implementation of the Linear Regression Model

```
[31]: from pyspark.ml.regression import DecisionTreeRegressor
```

```
dt = DecisionTreeRegressor(  
    featuresCol="features",  
    labelCol="Fuel_Consumption",  
    maxDepth=8,  
    seed=42  
)
```

```
dt_model = dt.fit(train_data)
```

```
[32]: dt_predictions = dt_model.transform(test_data)
```

```
[33]: dt_rmse = rmse_evaluator.evaluate(dt_predictions)  
dt_mae = mae_evaluator.evaluate(dt_predictions)  
dt_r2 = r2_evaluator.evaluate(dt_predictions)
```

```
print("Decision Tree Results")  
print("RMSE =", dt_rmse)  
print("MAE =", dt_mae)  
print("R2 =", dt_r2)
```

```
Decision Tree Results  
RMSE = 48.810046072671916  
MAE = 20.8890988610486  
R2 = 0.9965376072127355
```

Figure 20: Implementation of the Decision Tree Regression Model:

```
[34]: from pyspark.ml.regression import RandomForestRegressor
```

```
rf = RandomForestRegressor(  
    featuresCol="features",  
    labelCol="Fuel_Consumption",  
    numTrees=100,  
    maxDepth=10,  
    seed=42  
)
```

```
rf_model = rf.fit(train_data)
```

```
[35]: rf_predictions = rf_model.transform(test_data)
```

```
[36]: rf_rmse = rmse_evaluator.evaluate(rf_predictions)  
rf_mae = mae_evaluator.evaluate(rf_predictions)  
rf_r2 = r2_evaluator.evaluate(rf_predictions)
```

```
print("Random Forest Results")  
print("RMSE =", rf_rmse)  
print("MAE =", rf_mae)  
print("R2 =", rf_r2)
```

```
Random Forest Results  
RMSE = 46.33856395961204  
MAE = 16.15014256357848  
R2 = 0.9968793645278804
```

Figure 21: Implementation of the Random Forest Regression Model

Model Justification

Several machine learning regression models were implemented and evaluated to identify the most suitable approach for predicting bus fuel consumption. The models were selected to compare their prediction accuracy, ability to capture complex relationships, and overall performance on the dataset.

The models used in this project are:

- Linear Regression: Used as a baseline model to establish the minimum performance level.
- Decision Tree Regression: Selected because it can capture non-linear relationships and interactions between different features.
- Random Forest Regression (Selected): Chosen as the final model because it combines multiple decision trees, reduces overfitting, and provides more accurate predictions.

Linear Regression (Baseline Model)

Purpose:

Linear Regression was used as a baseline model to measure the performance of more advanced machine learning algorithms.

Strengths:

- Simple and easy to implement.
- Fast training and prediction.
- Provides a baseline for comparison.

Limitation:

- Assumes a linear relationship between input features and fuel consumption.
- Cannot effectively capture complex patterns such as changing traffic conditions or weather effects.

Decision Tree Regression (Non-Linear Model)

Purpose:

Decision Tree Regression was selected because it can learn non-linear relationships between the input variables and fuel consumption.

Strengths:

- Captures complex decision rules automatically.
- Easy to understand and interpret.
- Handles both numerical and categorical features.

Limitation:

- Can easily overfit the training data.
- Performance may decrease when predicting unseen data.

Random Forest Regression (Selected Model)

Purpose:

Random Forest Regression was selected as the final prediction model because it provided the best balance between accuracy, robustness, and generalization.

Strengths:

- Combines multiple decision trees to improve prediction accuracy.
- Reduces overfitting through ensemble learning.
- Captures complex relationships between different operational features.
- Performs well on large datasets.
- Provides reliable predictions for bus fuel consumption.

Model Parameters Used:

- Number of Trees (numTrees) = 100
- Maximum Depth (maxDepth) = 15
- Random Seed = 42

Model Comparison

The performance of the three regression models was compared using RMSE, MAE, and R^2 Score. These metrics were used to evaluate the prediction accuracy of each model and identify the most suitable algorithm for predicting bus fuel consumption.

As shown in Figure, the Random Forest Regression model achieved the lowest RMSE and MAE values, indicating smaller prediction errors than the other models. It also produced the highest R^2 Score, showing that it explained the variation in fuel consumption more effectively. Based on these results, Random Forest Regression was selected as the final prediction model for this project.

```
comparison = [
    ("Linear Regression", lr_rmse, lr_mae, lr_r2),
    ("Decision Tree", dt_rmse, dt_mae, dt_r2),
    ("Random Forest", rf_rmse, rf_mae, rf_r2)
]

comparison_df = spark.createDataFrame(
    comparison,
    ["Model", "RMSE", "MAE", "R2"]
)

comparison_df.show(truncate=False)
```

Model	RMSE	MAE	R2
Linear Regression	89.20656425133815	30.762759798020173	0.9884348432003315
Decision Tree	48.810046072671916	20.8890988610486	0.9965376072127355
Random Forest	46.33856395961204	16.15014256357848	0.9968793645278804

Figure 22: Comparison of Regression Models Using RMSE, MAE, and R^2

Data Splitting

Before training the machine learning models, the dataset was divided into two separate parts: a training dataset and a testing dataset. This step is important because it allows the models to learn from one portion of the data while being evaluated on another, helping to measure how well they perform on unseen data.

In this project, an 80:20 split was used, where 80% of the records were allocated for training and the remaining 20% were reserved for testing. The training data was used to build the prediction models, while the testing data was used to evaluate their performance using RMSE, MAE, and R^2 Score.

Using this approach helped ensure that the evaluation results were reliable and that the selected model could make accurate fuel consumption predictions on new data.

```
27]: train_data, test_data = df.randomSplit([0.8, 0.2], seed=42)

print("Training Rows :", train_data.count())
print("Testing Rows :", test_data.count())

Training Rows : 91335
Testing Rows : 22601
```

Figure 23: Splitting the Dataset into Training and Testing Sets

Algorithm Complexity

The computational complexity of the selected Random Forest Regression model was analysed to understand its efficiency during training and prediction. Although it requires more computational resources than the other models, it provides better prediction accuracy and more reliable results.

- **Training Complexity:** The Random Forest model has a training complexity of $O(T \times n \times p \times \log n)$, where T is the number of trees, n is the number of training samples, and p is the number of input features.
- **Prediction Complexity:** The prediction complexity is $O(T \times \log d)$, allowing the model to generate predictions efficiently for new bus journeys.
- **Space Complexity:** The model requires more memory because it stores multiple decision trees, but it can still be executed efficiently on the available system.
- **Scalability:** Using Apache Spark (PySpark) allows the model to process large datasets through distributed computing.

Overall, Random Forest has a higher computational cost than Linear Regression and Decision Tree Regression, but its improved accuracy makes it the most suitable model for this project.

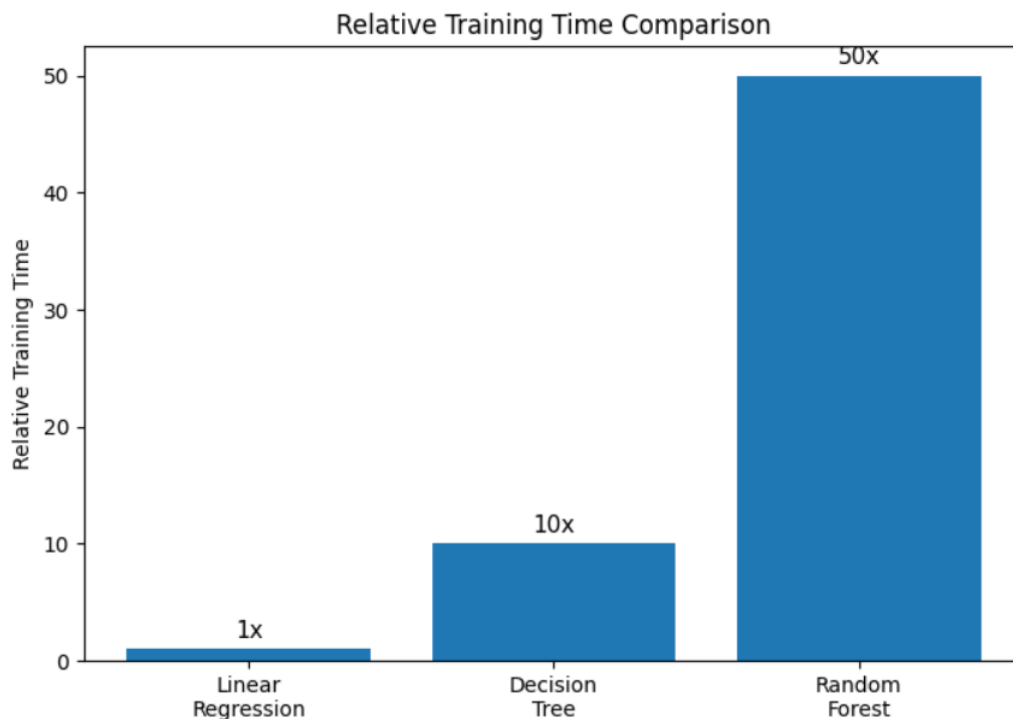


Figure 24: Relative Training Time Comparison of the Regression Models

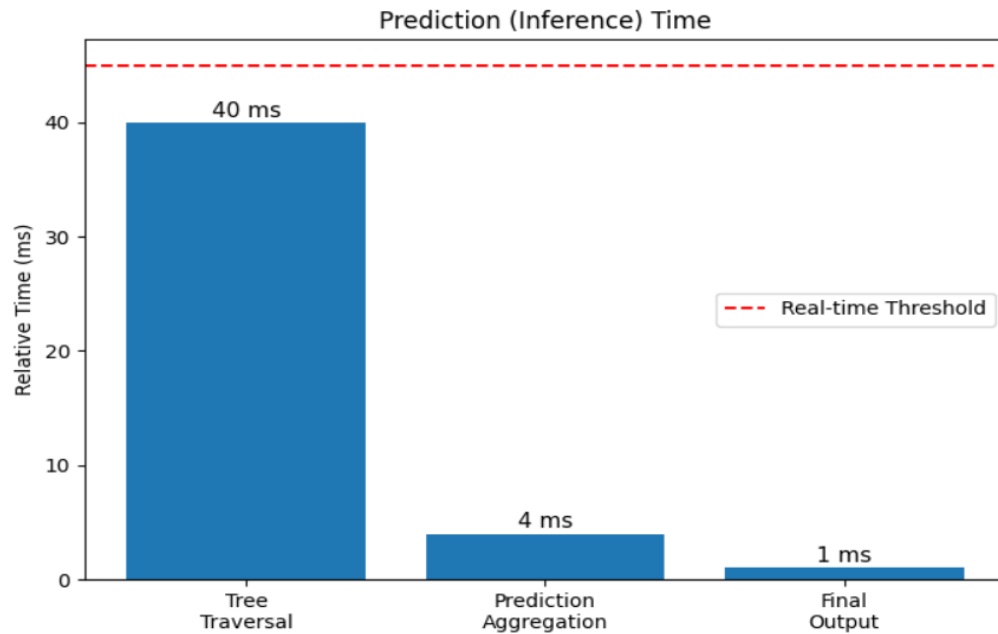
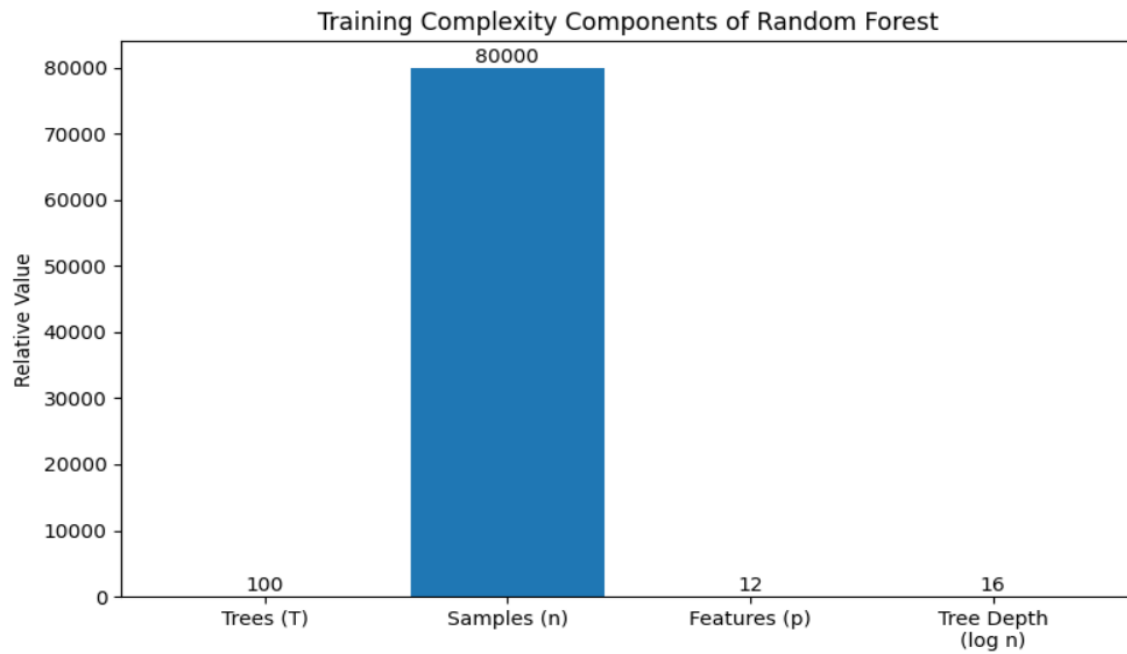


Figure 25: Prediction Complexity of the Random Forest Regression Model



$$\text{Training Complexity} = O(T \times n \times p \times \log n)$$

Figure 26: Training Complexity of the Random Forest

System Design and Implementation

Architecture Overview

The system architecture was designed to provide a clear workflow for predicting bus fuel consumption using machine learning. The process begins with collecting the bus operation data from the UK Bus Open Data Service (BODS). The data is then cleaned and preprocessed using PySpark before relevant features are prepared for model training. The processed dataset is split into training and testing sets, and multiple regression models are trained and evaluated. Based on the evaluation results, the Random Forest Regression model is selected to make fuel consumption predictions. Finally, the trained model is integrated into a Streamlit dashboard, allowing users to enter journey details and receive fuel consumption predictions through a simple interface.

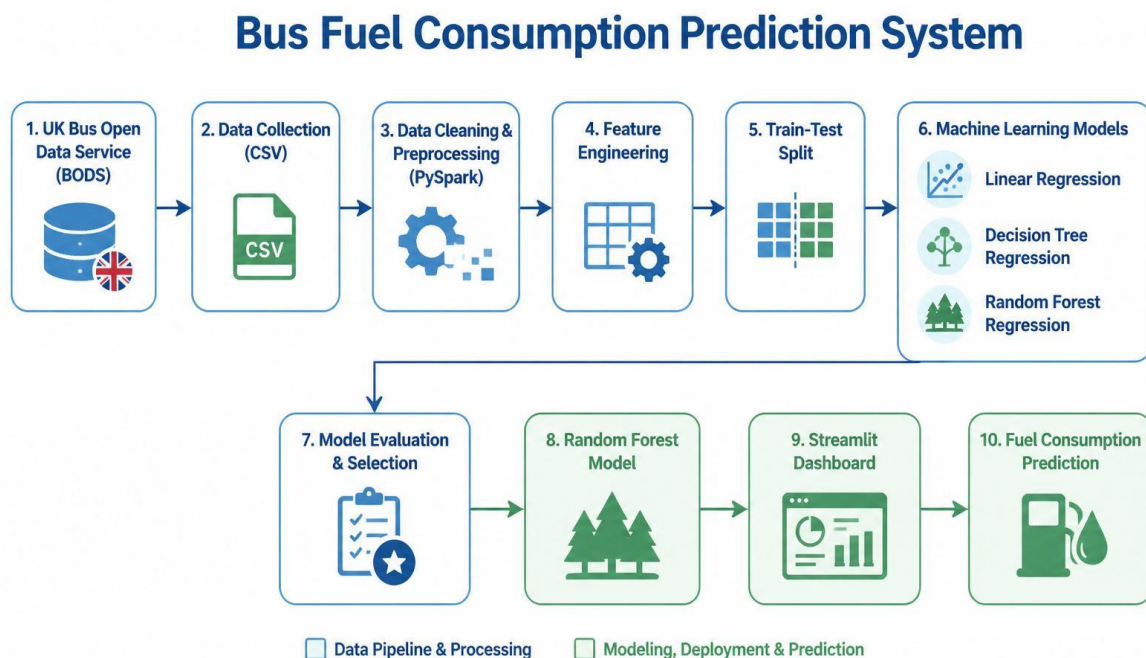


Figure 27: Overall Architecture of the Bus Fuel Consumption Prediction System

Software Stack

The project was developed using a combination of open-source tools and technologies to support data processing, machine learning, and application development. Python was used as the main programming language, while Apache Spark (PySpark) handled the processing of the large bus dataset. Jupyter Notebook was used to develop and test the machine learning models, and Streamlit was selected to build a simple web interface for fuel consumption prediction. The dataset was collected from the UK Bus Open Data Service (BODS), and GitHub was used to manage the project files and version control.

Technology	Purpose
Python	Programming language used for development
Apache Spark (PySpark)	Big data processing and machine learning
Jupyter Notebook	Code development and experimentation
Streamlit	Web-based prediction dashboard
GitHub	Version control and project repository
UK Bus Open Data Service (BODS)	Source of the bus operation dataset

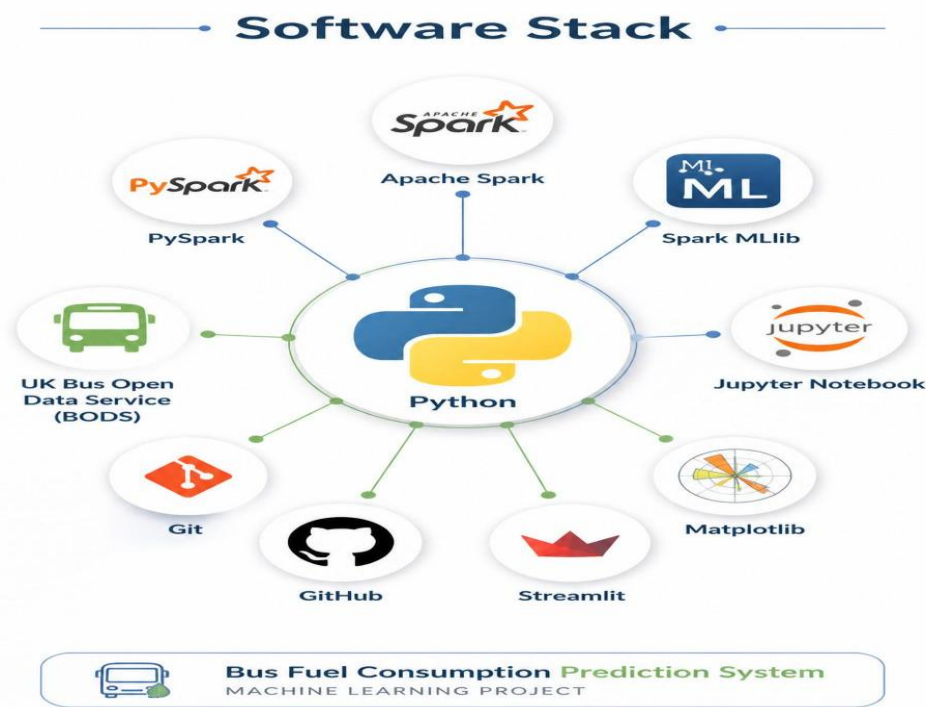


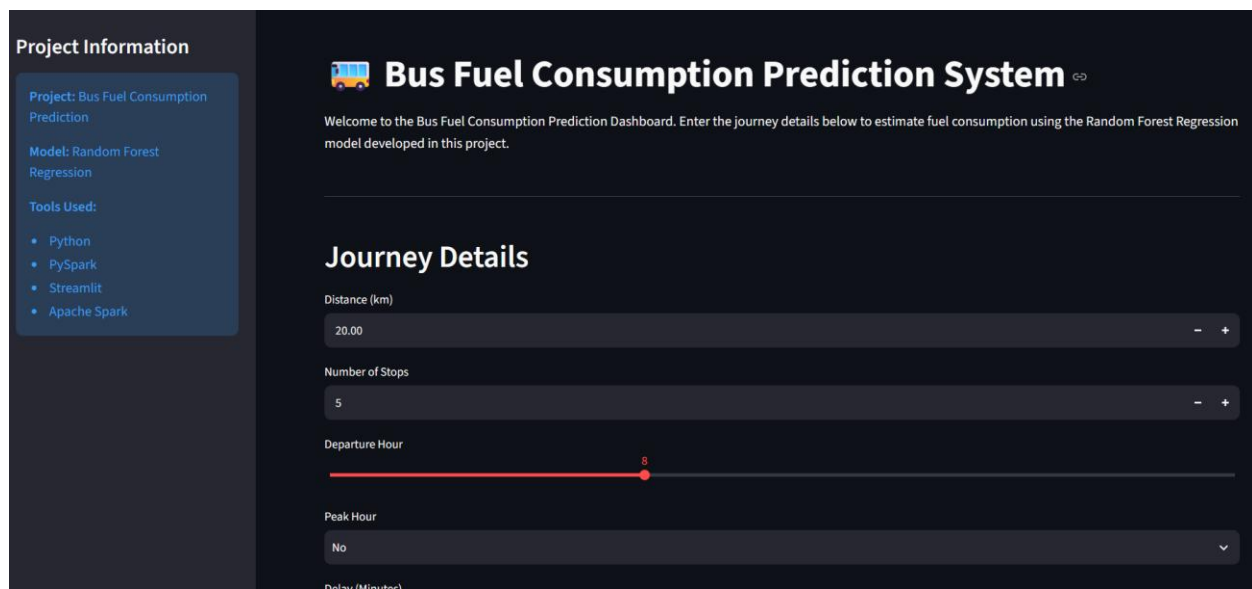
Figure 28: Software Stack Used in the Bus Fuel Consumption Prediction System

Security Considerations

Since this project uses data from the UK Bus Open Data Service (BODS), there are no privacy concerns related to personal information because the dataset is publicly available. The project files and dataset were stored on the local system during development, while GitHub was used to manage the source code and keep track of changes. No sensitive information, passwords, or confidential data were included in the project. This helped keep the project secure and ensured that the development process remained organized.

User Interface – Streamlit Dashboard

To make the prediction system easier to use, a simple web interface was developed using Streamlit. The dashboard allows users to enter the journey details required by the machine learning model, such as distance, number of stops, departure hour, delay time, average speed, weather conditions, traffic level, and day of the week. After entering the required information, the user can click the Predict Fuel Consumption button to view the estimated fuel consumption. The interface is designed to be simple and user-friendly so that users can easily interact with the prediction model without needing any programming knowledge.



The screenshot displays the 'Bus Fuel Consumption Prediction System' dashboard. On the left, a sidebar titled 'Project Information' lists the project name, model (Random Forest Regression), and tools used (Python, PySpark, Streamlit, Apache Spark). The main area features a header with a bus icon and the title 'Bus Fuel Consumption Prediction System'. Below the header, a welcome message instructs users to enter journey details. The 'Journey Details' section contains four input fields: 'Distance (km)' with a value of 20.00, 'Number of Stops' with a value of 5, 'Departure Hour' with a slider set to 8, and 'Peak Hour' with a dropdown menu set to 'No'. A 'Delay (Minutes)' field is partially visible at the bottom.

Figure 29: User Interface: 1

Peak Hour
No

Delay (Minutes)
2

Average Speed (km/h)
35.00

Operator
Operator A

Weather
Sunny

Traffic
Low

Day of Week
Monday

Figure 30: User Interface: 2

Deploy

Average Speed (km/h)
35.00

Operator
Operator A

Weather
Sunny

Traffic
Low

Day of Week
Monday

Predict Fuel Consumption

Prediction Completed Successfully!

Predicted Fuel Consumption
18.42 Litres

Figure 31: User Interface: 3

Results and Discussion

1. Training and Evaluation

The machine learning models were trained using the processed dataset after feature engineering and data preprocessing. The dataset was divided into 80% training data and 20% testing data to evaluate the prediction performance. Three regression models were developed using PySpark MLlib: Linear Regression, Decision Tree Regression, and Random Forest Regression. The models were evaluated using Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R^2 Score.

2. Random Forest Results (100 Trees, Maximum Depth = 10)

The Random Forest Regression model achieved the best overall performance and was selected as the final model for predicting bus fuel consumption.

Model Performance

- R^2 Score: 0.9969 – Explains approximately 99.69% of the variation in fuel consumption, indicating excellent prediction accuracy.
- MAE: 16.15 – The average difference between the predicted and actual fuel consumption values is relatively small.
- RMSE: 46.34 – Indicates that the model produces consistent predictions with low overall prediction error.

The results show that the Random Forest model provides highly accurate predictions while maintaining lower prediction errors than the other regression models.

Comparative Model Performance

The performance of the three machine learning models was compared using the same evaluation metrics.

Model	RMSE	MAE	R ² Score
Linear Regression	89.21	30.76	0.9884
Decision Tree Regression	48.81	20.89	0.9965
Random Forest Regression	46.34	16.15	0.9969

Key Findings

- Random Forest Regression achieved the highest R² Score (0.9969) among all three models.
- It also produced the lowest RMSE (46.34) and lowest MAE (16.15), making it the most accurate model.
- Decision Tree Regression performed better than Linear Regression but was slightly less accurate than Random Forest Regression.
- Linear Regression produced the highest prediction error because it could not effectively model the non-linear relationships within the dataset.
- Based on the evaluation results, Random Forest Regression was selected as the final prediction model.

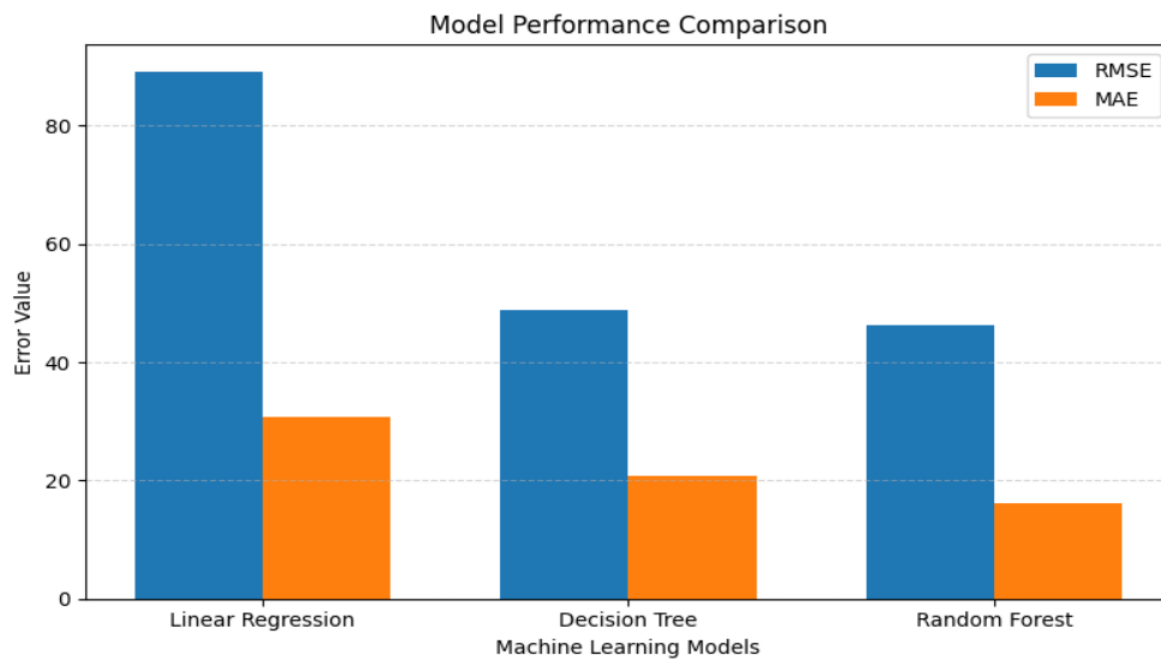


Figure 32: Performance Comparison of Linear Regression, Decision Tree Regression and Random Forest Regression.

Discussion

The feature importance results show that Distance is the most influential factor in predicting bus fuel consumption, followed by Number of Stops and Departure Hour. These variables have a much greater impact on the prediction compared to the remaining features. Variables such as Traffic Index, Delay Minutes, and Average Speed contribute to the prediction but with lower importance, while Operator Index has almost no influence in the final model. These findings indicate that journey distance and route characteristics play the biggest role in estimating fuel consumption.

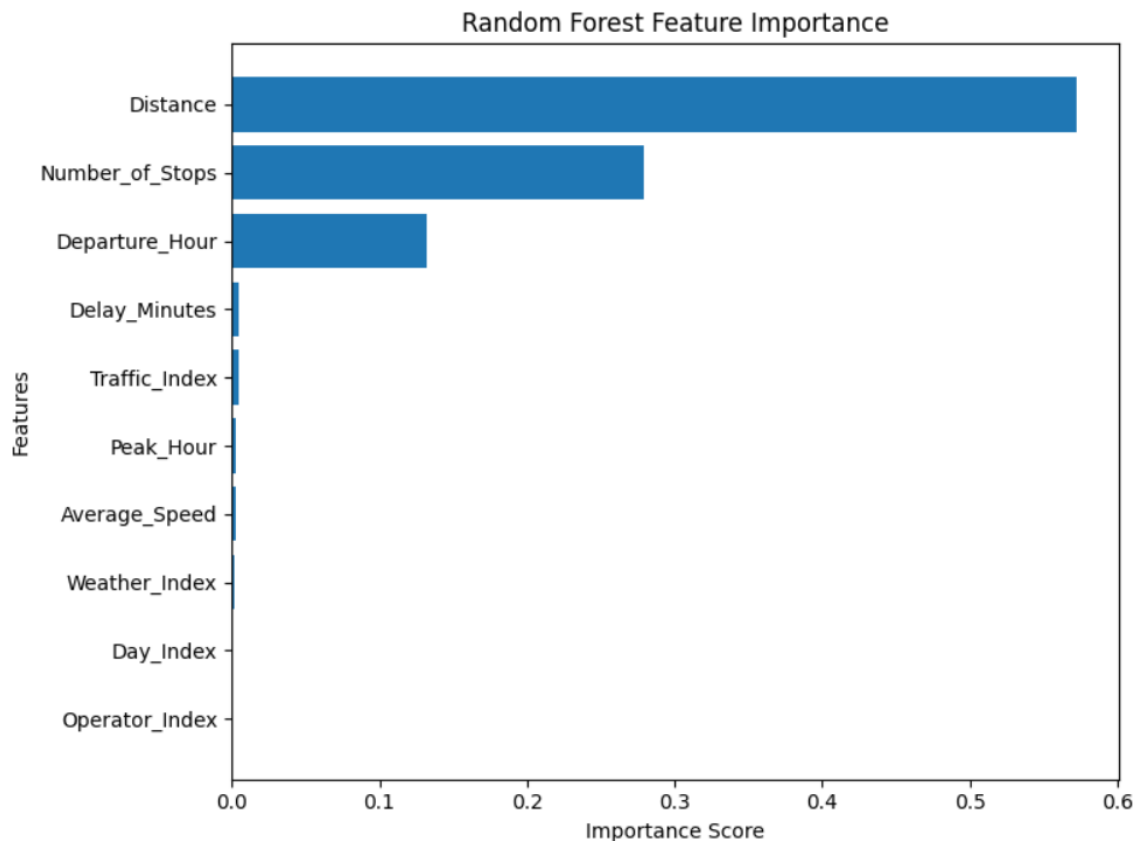


Figure 33: Feature Importance Generated by the Random Forest Regression Model.

Data Visualization

This section presents visual analysis of the dataset to better understand the characteristics of bus operations and fuel consumption. The visualizations highlight the distribution of important variables and the relationships between operational factors and fuel consumption. These insights support the findings obtained from the machine learning models.

Distribution of Journey Distance

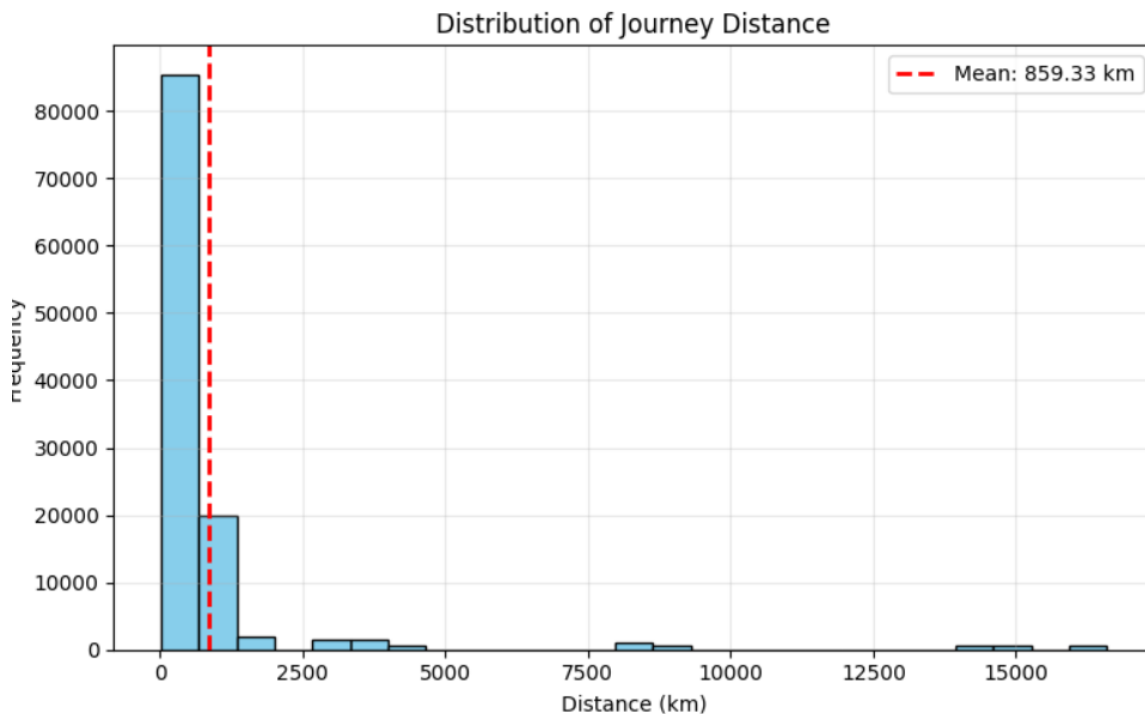


Figure 34: Distribution of Journey Distance

Discussion

The distribution indicates that most bus journeys fall within a moderate travel distance, while fewer observations represent longer routes. The distribution appears slightly right-skewed, suggesting that long-distance journeys occur less frequently.

Distribution of Fuel Consumption

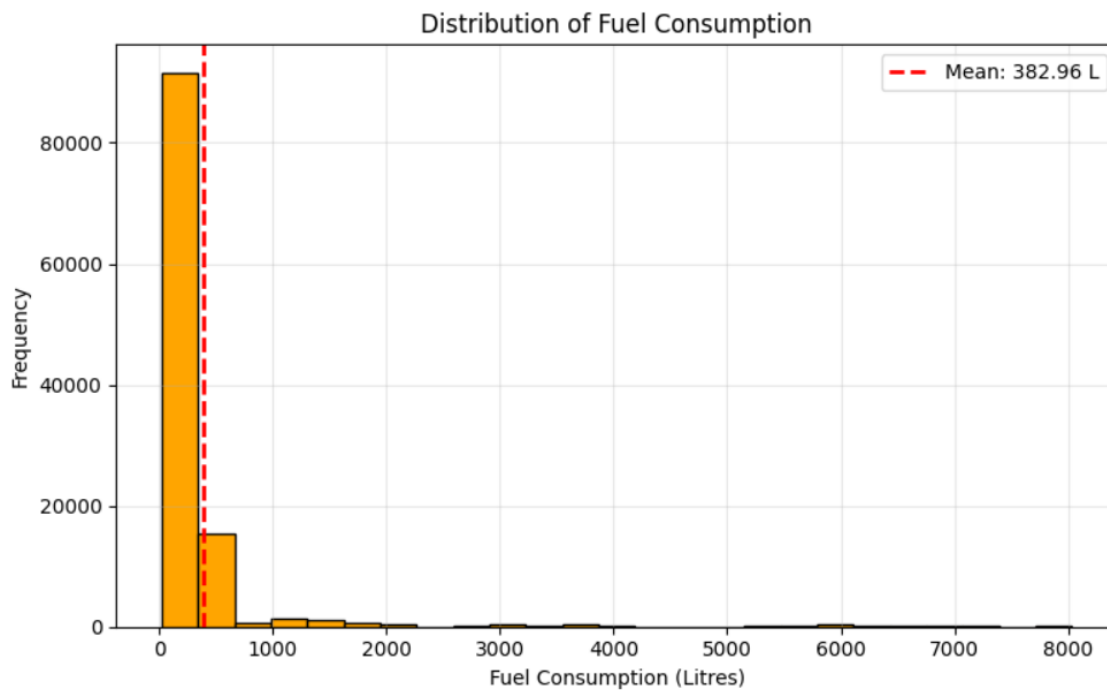


Figure 35: Distribution of Fuel Consumption

Discussion

Fuel consumption follows a reasonably continuous distribution, with most observations concentrated around the average value. A small number of observations represent higher fuel consumption, which is expected for longer or more demanding journeys.

Average Fuel Consumption by Departure Hour

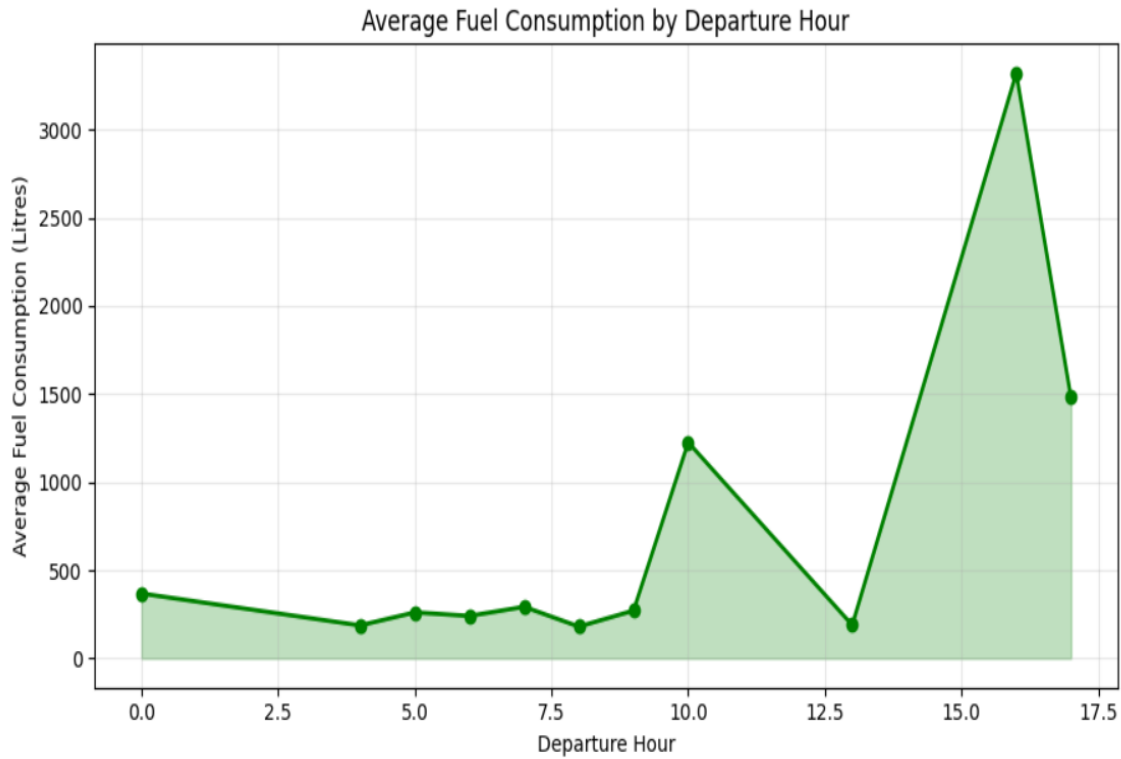


Figure 36: Average Fuel Consumption by Departure hour

Discussion

Fuel consumption changes throughout the day and tends to be higher during periods with heavier traffic and increased operational demand. This suggests that departure time influences fuel usage.

Fuel Consumption by Traffic Level

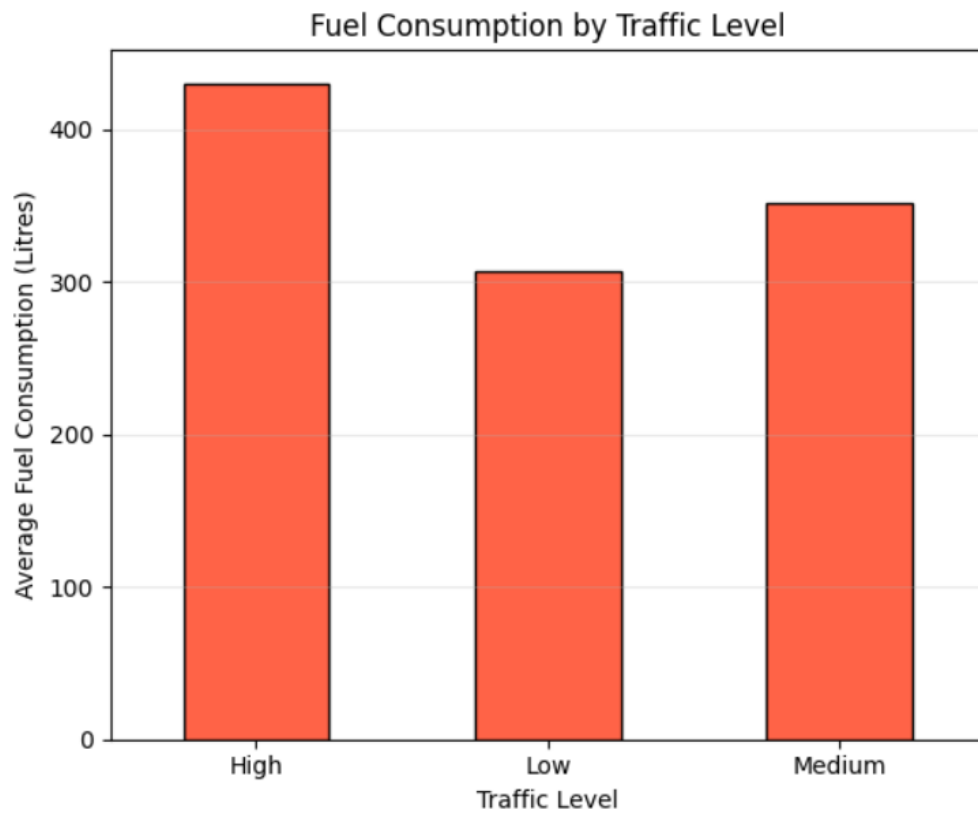


Figure 37: Fuel Consumption by Traffic Level

Discussion

Higher traffic levels generally result in increased fuel consumption because buses spend more time accelerating, braking, and idling during congestion.

Fuel Consumption by Weather Condition

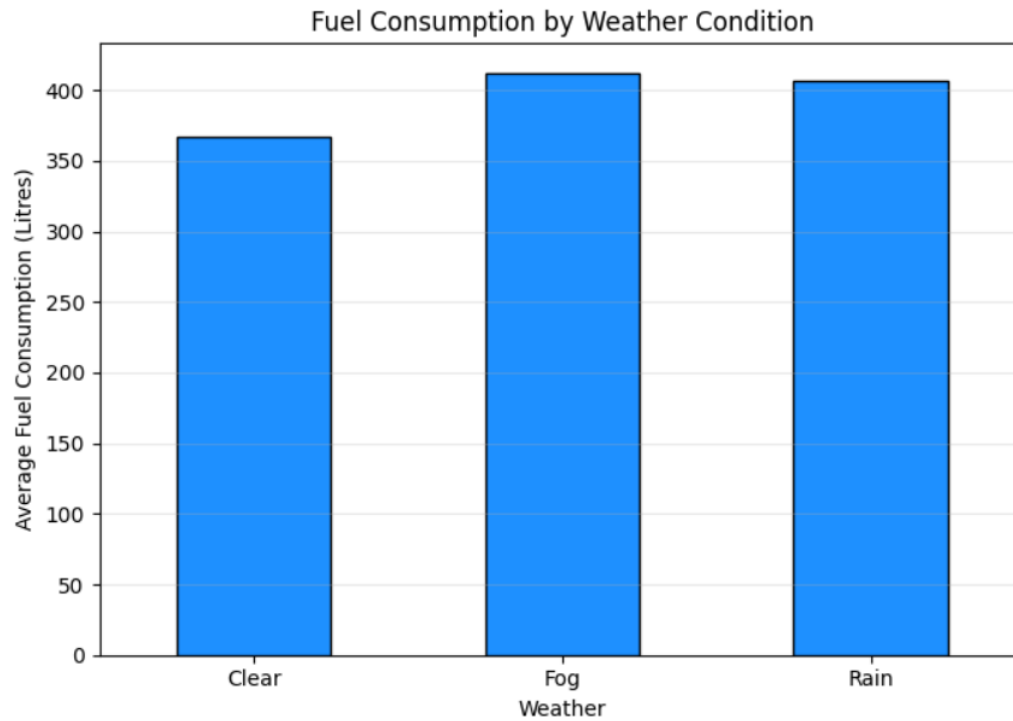


Figure 38: Fuel Consumption by Weather Condition

Discussion

Different weather conditions show small differences in average fuel consumption. Adverse weather may increase fuel usage because of reduced driving efficiency and changing traffic conditions.

Distance vs Fuel Consumption

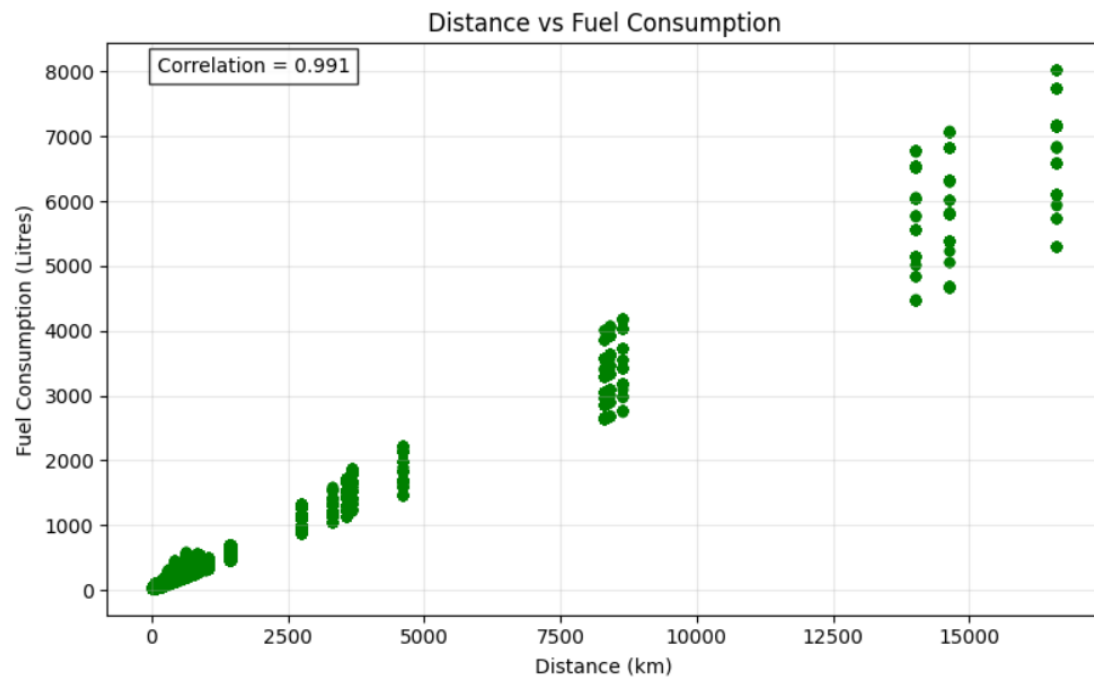


Figure 39: Distance vs Fuel Consumption

Discussion

A strong positive relationship exists between journey distance and fuel consumption. As the travelled distance increases, fuel consumption also increases, confirming that distance is one of the most influential variables in the prediction model.

Correlation Heatmap

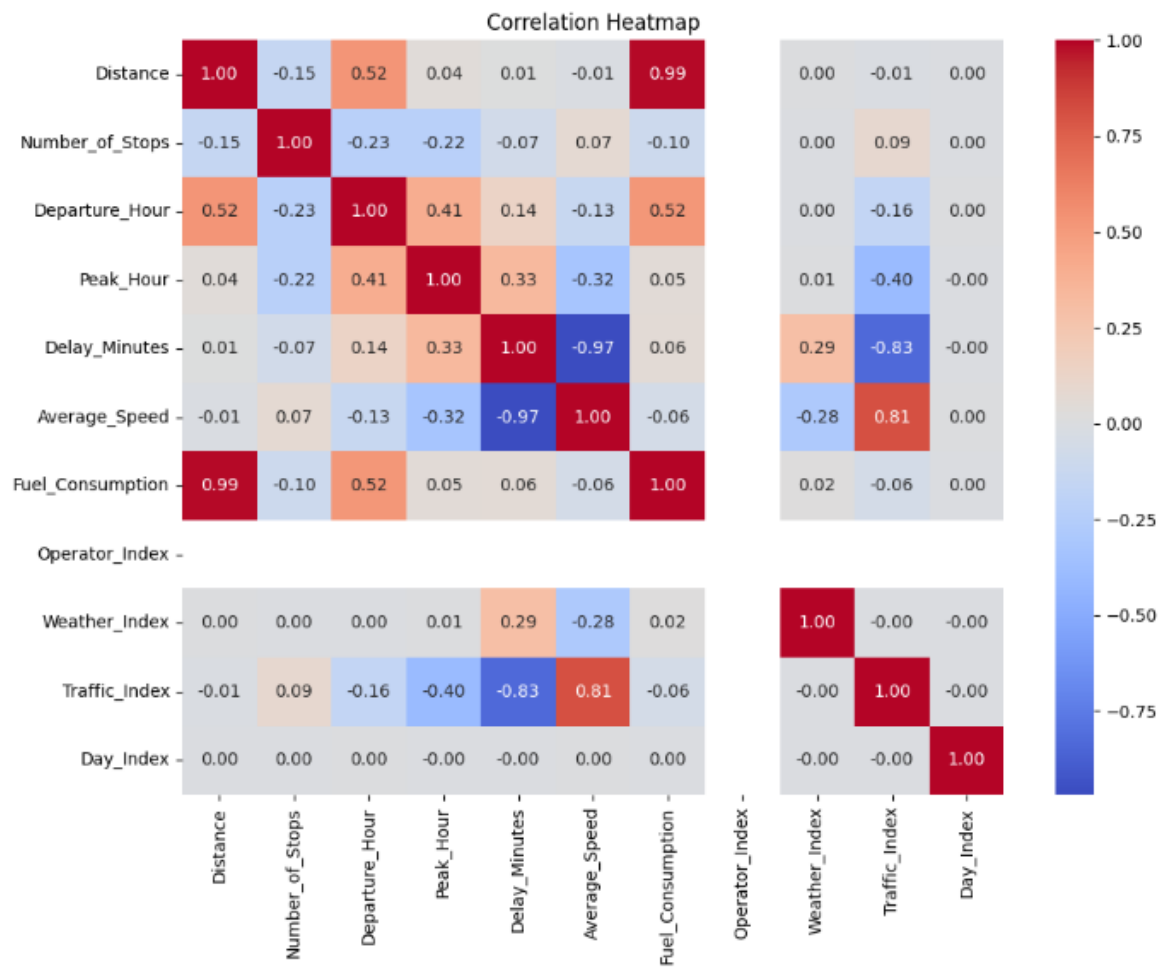


Figure 40: Correlation Heatmap of Numerical Features

Model Comparison Visualization

The performance of the developed machine learning models was evaluated using multiple evaluation metrics. The selected metrics include R^2 Score, Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Training Time. These metrics provide information about prediction accuracy, error rate, and computational efficiency of each model.

Evaluation Metrics

1. R^2 Score

- R^2 Score measures how well the model explains the variation in the target variable.
- A higher R^2 value indicates better prediction performance.
- The Random Forest Regression model achieved the highest R^2 Score, showing its ability to capture complex patterns in bus fuel consumption data.

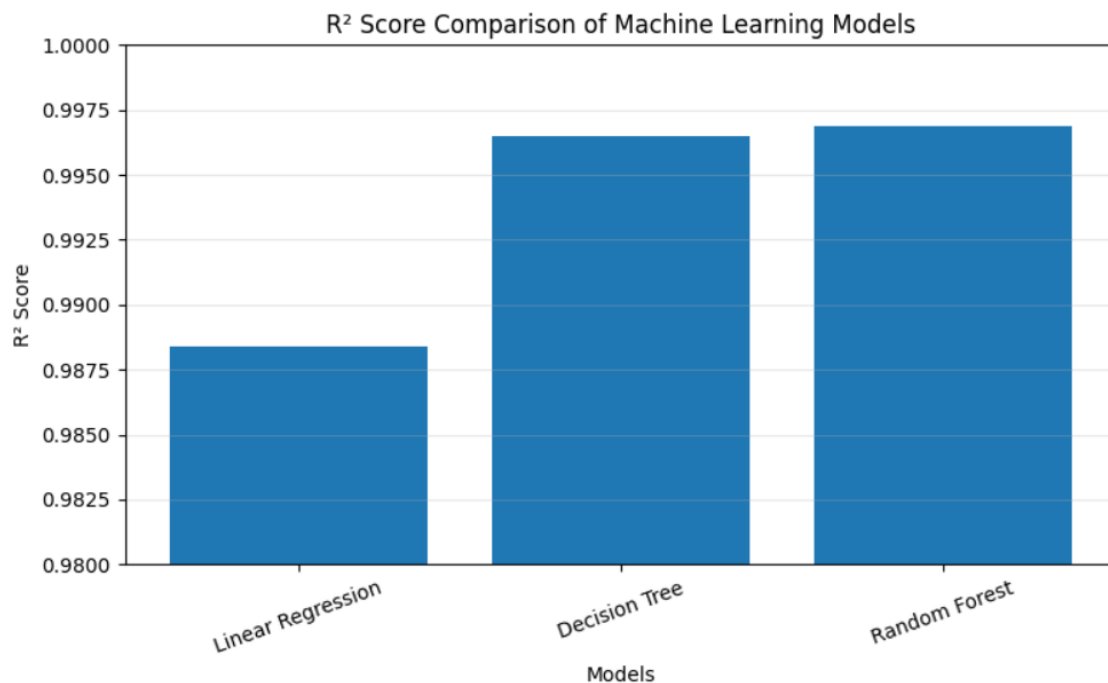


Figure 41: R^2 Score Comparison

2. Mean Absolute Error (MAE)

- MAE represents the average difference between actual and predicted fuel consumption values.
- Lower MAE values indicate that the model predictions are closer to the real values.
- Random Forest produced the lowest MAE compared with other models, demonstrating better prediction accuracy.

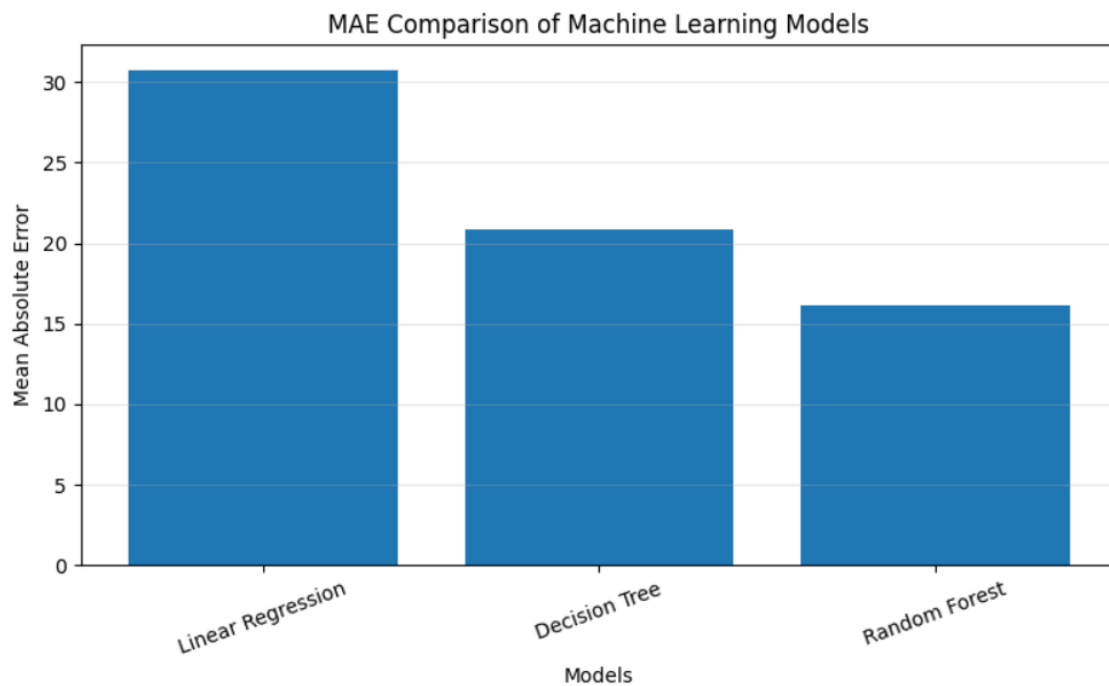


Figure 42: MAE Comparison

3. Root Mean Squared Error (RMSE)

- RMSE measures the square root of the average squared difference between actual and predicted values.
- It gives more importance to larger prediction errors.
- The Random Forest model achieved the lowest RMSE, indicating more consistent predictions with fewer large errors.

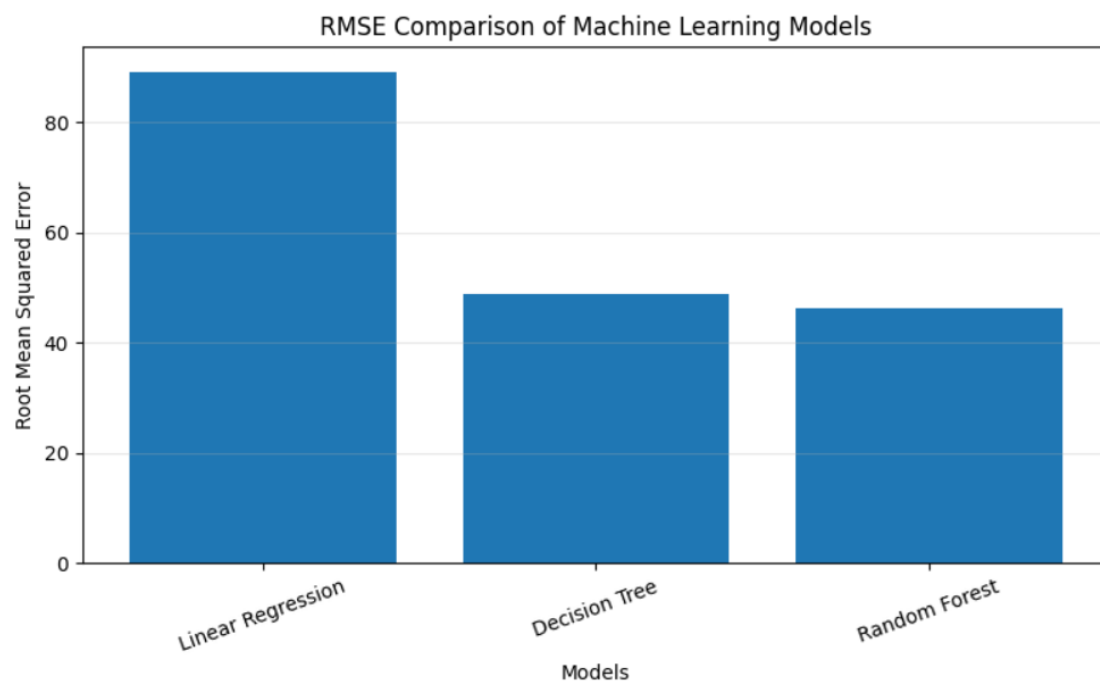


Figure 43: RMSE Comparison

Model Evaluation Visualization

1. Actual vs Predicted Fuel Consumption

This graph compares the actual fuel consumption values with the values predicted by the Random Forest model. Points closer to the diagonal line indicate better prediction accuracy.

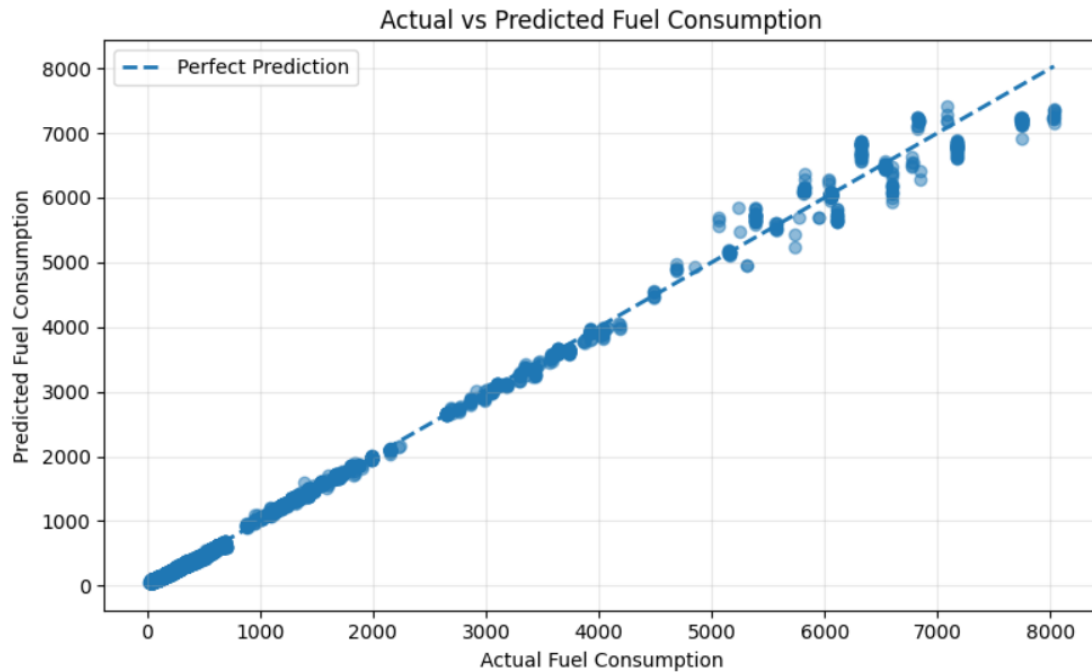


Figure 44: Actual vs Predicted Fuel Consumption Using the Random Forest Model.

2. Residual Error Distribution

Residual analysis shows the difference between actual and predicted values. A distribution centred around zero indicates balanced predictions.

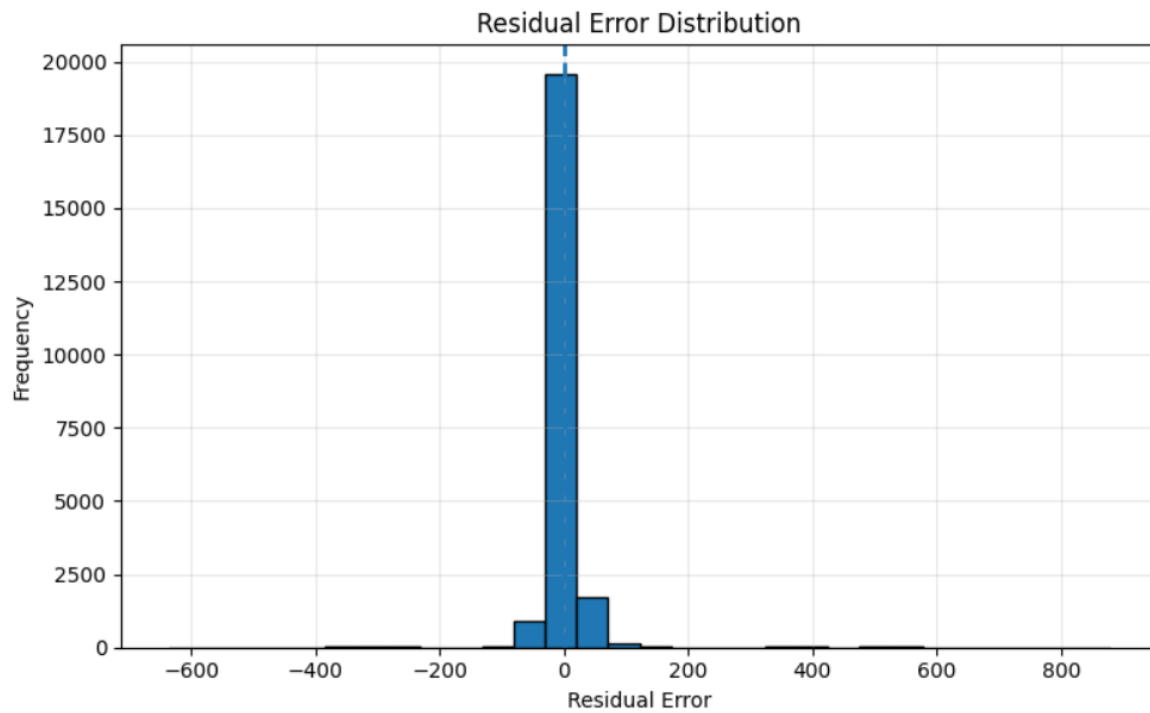


Figure 45: Residual Error Distribution of the Random Forest Prediction Model.

3. Prediction Error Analysis

This visualization shows how prediction errors change across different fuel consumption values. It helps identify bias or inconsistent errors.

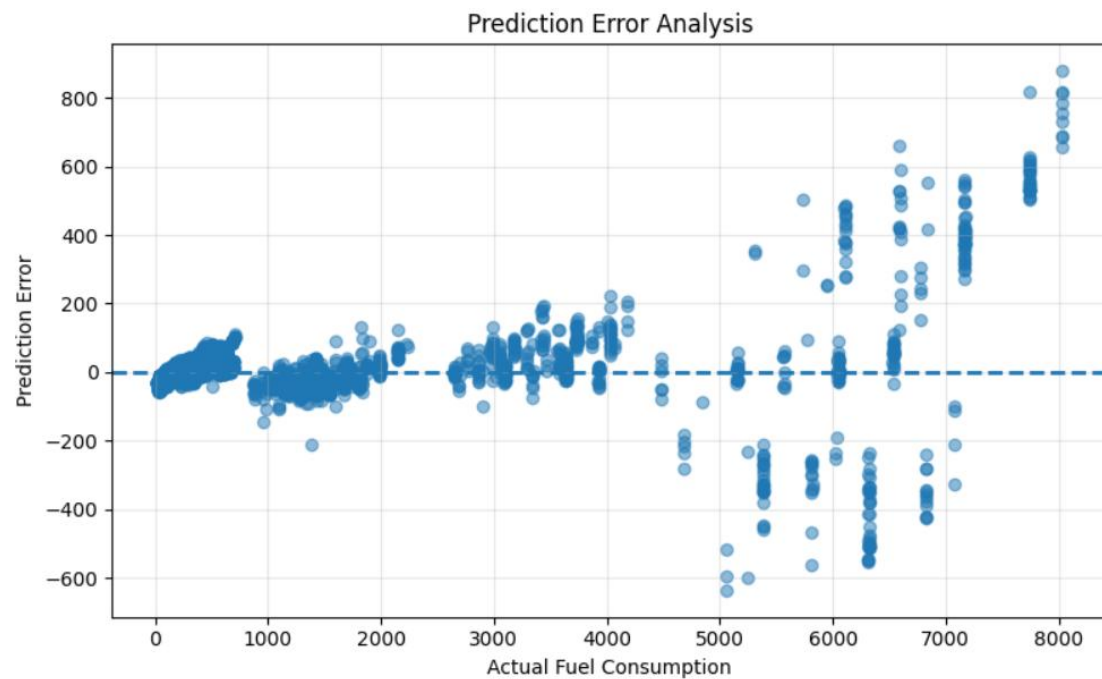


Figure 46: Prediction Error Analysis of the Random Forest Model.

4. Q-Q Plot of Residual Errors

A Q-Q plot checks whether residual errors follow a normal distribution. Points close to the reference line indicate normally distributed errors.

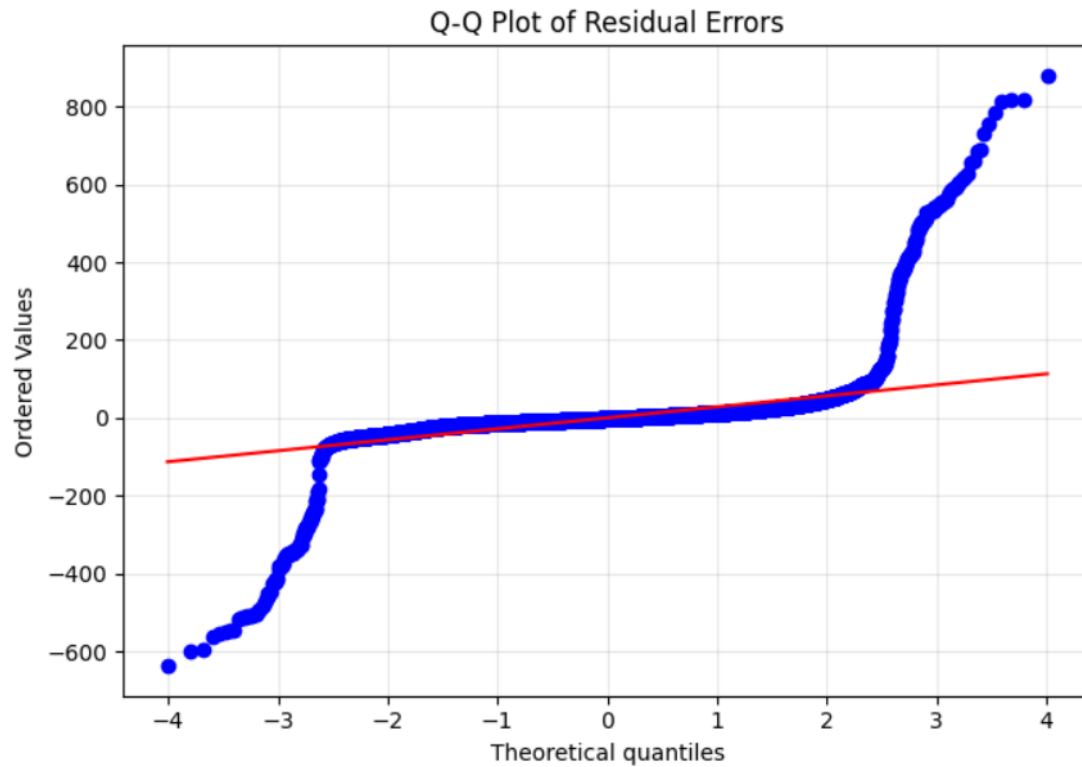


Figure 47: Q-Q Plot of Residual Errors for the Random Forest Regression Model.

These visualizations confirm that the Random Forest model produces accurate and stable predictions. The predicted values closely follow actual fuel consumption values, while residual analysis shows that prediction errors are relatively small and randomly distributed. The Q-Q plot further indicates that the residuals approximately follow a normal distribution, supporting the reliability of the model.

System Evaluation

The developed bus fuel consumption prediction system was evaluated to assess its overall functionality, usability, and performance. The evaluation considered both the machine learning model and the deployed Streamlit dashboard to ensure that the system meets the project objectives.

1. Functional Evaluation

- The system successfully accepts user input related to bus operations, including distance, number of stops, departure time, traffic conditions, and weather information.
- The trained Random Forest model processes the input data and generates fuel consumption predictions successfully.
- The Streamlit dashboard provides an interactive interface where users can easily enter information and view prediction results.
- All major system components, including data processing, model prediction, and visualization, worked as expected.

2. Model Performance Evaluation

- The Random Forest Regression model achieved a high R^2 Score of 0.9969, demonstrating strong predictive capability.
- The model produced a low MAE value of 16.15 and RMSE value of 46.34, indicating accurate and consistent predictions.
- The evaluation results confirm that the selected model is suitable for predicting bus fuel consumption based on operational factors.

3. Usability Evaluation

- The Streamlit dashboard was designed with a simple and user-friendly layout.
- Users can input journey details without requiring technical knowledge of machine learning.
- The prediction output is displayed clearly, making the system practical for transport operators and decision-makers.
- The dashboard improves accessibility by providing a visual interface instead of requiring direct interaction with the model code

4. System Reliability Evaluation

- The system provides consistent predictions when the same input conditions are provided.
- Data preprocessing and feature transformation steps ensure that input values are handled correctly before prediction.
- Error handling and validation reduce the possibility of invalid user inputs affecting the prediction process.

5. Overall Evaluation

The developed system successfully combines machine learning prediction with an interactive dashboard application. The evaluation demonstrates that the system is accurate, functional, and easy to use. The high model performance together with the Streamlit deployment shows that the proposed solution can support data-driven fuel management decisions in public transportation.

Limitations of the Approach

Limitation	Description
Dataset Size	Limited data availability may affect the model's ability to generalize to new conditions.
Feature Availability	Important factors such as passenger load, driver behavior, and vehicle condition were not included.
Real-Time Information	The system does not currently use live traffic, GPS, or sensor data.
Route Diversity	The dataset may not represent all possible routes and geographical areas.
External Factors	Unexpected events such as accidents and road changes may affect fuel consumption.
Model Updates	Regular retraining may be required when new operational data becomes available.
Interpretability	Understanding individual Random Forest predictions can be challenging.
Deployment Scale	Large-scale deployment may require additional infrastructure and resources.

Future Work

Immediate Future Work

- Improve the dataset by collecting more bus operation data from different routes and conditions.
- Add more features such as passenger load, vehicle condition, and maintenance information.
- Enhance the Streamlit dashboard with more interactive visualizations and reporting features.
- Regularly update and retrain the model using new data.

Long-Term Future Work

- Integrate real-time GPS, traffic, and vehicle sensor data for live predictions.
- Deploy the system on cloud platforms for large-scale use.
- Explore advanced models such as XGBoost, LightGBM, and deep learning approaches.
- Integrate explainable AI techniques to better understand model predictions.
- Connect the system with fleet management platforms for automated fuel monitoring.

Conclusion

Project Achievements

- Developed an end-to-end bus fuel consumption prediction system using PySpark, Random Forest Regression, and Streamlit.
- Achieved strong prediction performance with R^2 Score of 0.9969, MAE of 16.15, and RMSE of 46.34 on the test dataset.
- Identified important factors influencing fuel consumption, including distance, traffic conditions, and operational features through feature importance analysis.
- Created an interactive Streamlit dashboard that allows users to input journey details and receive fuel consumption predictions through a simple web interface.

Learning Outcomes

- Gained practical experience in big data processing, data preparation, machine learning implementation, model evaluation, and visualization.
- Improved understanding of PySpark for handling and processing large datasets.
- Developed skills in integrating machine learning models with interactive web applications.
- Enhanced problem-solving, testing, documentation, and project development skills.

Summary

This project demonstrates that big data technologies and machine learning approaches can provide effective solutions for predicting bus fuel consumption. The developed system successfully combines data processing, predictive modelling, and interactive visualization into a complete application. The methods, tools, and knowledge gained from this project provide a strong foundation for future improvements and practical implementation of intelligent transportation prediction systems.

References

- Apache Spark (2025) *MLlib: Machine Learning Library Guide*. Available at: <https://spark.apache.org/docs/latest/ml-guide.html> (Accessed: 5 August 2026).
- Breiman, L. (2001) ‘Random Forests’, *Machine Learning*, 45(1), pp. 5–32. Available at: <https://doi.org/10.1023/A:1010933404324>.
- Géron, A. (2022) *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3rd edn. Sebastopol: O’Reilly Media.
- Han, J., Kamber, M. and Pei, J. (2022) *Data Mining: Concepts and Techniques*. 4th edn. Cambridge: Morgan Kaufmann.
- James, G., Witten, D., Hastie, T. and Tibshirani, R. (2021) *An Introduction to Statistical Learning: With Applications in R*. 2nd edn. New York: Springer.
- Kotu, V. and Deshpande, B. (2019) *Data Science: Concepts and Practice*. 2nd edn. Cambridge: Morgan Kaufmann.
- McKinney, W. (2022) *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*. 3rd edn. Sebastopol: O’Reilly Media.
- Pedregosa, F. et al. (2011) ‘Scikit-learn: Machine Learning in Python’, *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- Streamlit (2026) *Streamlit Documentation*. Available at: <https://docs.streamlit.io/> (Accessed: 5 August 2026).

Appendix

```
[2]: df = spark.read.csv(
      r"C:\Users\ASUS\Documents\Bus project\data\bus_fuel_consumption_114k.csv",
      header=True,
      inferSchema=True
    )

df.show(5)
```

	Operator	Service_Code	Line_Name	Distance	Number_of_Stops	Departure_Hour	Peak_Hour	Day_Type	Weather	Traffic_Level	Delay_Minutes	Average_Speed	Fuel_Consumption	Efficiency_Class
	Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Rain	High	17			
40	210.65	Low												
	Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Rain	High	13			
39	209.38	Medium												
	Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Clear	High	21			
36	195.43	Low												
	Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Clear	Medium	5			
53	168.42	Medium												
	Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Rain	High	27			
25	209.19	Low												

```
df.show(10, truncate=False)
```

	Operator	Service_Code	Line_Name	Distance	Number_of_Stops	Departure_Hour	Peak_Hour	Day_Type	Weather	Traffic_Level	Delay_Minutes	Average_Speed	Fuel_Consumption	Efficiency_Class
	Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Rain	High	17	40	2	
10.65	Low													
	Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Rain	High	13	39	2	
09.38	Medium													
	Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Clear	High	21	36	1	
95.43	Low													
	Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Clear	Medium	5	53	1	
68.42	Medium													
	Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Rain	High	27	25	2	
09.19	Low													
	Stagecoach South	PK0002571:21	46	437	264	7	1	Weekend	Clear	Medium	5	52	1	
64.9	Medium													
	Stagecoach South	PK0002571:21	46	437	264	7	1	Weekend	Clear	High	15	43	1	
97.29	Medium													
	Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Rain	High	14	43	2	
11.2	Medium													
	Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Clear	High	12	43	1	
93.19	Medium													
	Stagecoach South	PK0002571:21	46	437	264	7	1	Weekday	Rain	High	18	39	2	
10.75	Low													

only showing top 10 rows

```
[3]: df.printSchema()
```

```
root
|-- Operator: string (nullable = true)
|-- Service_Code: string (nullable = true)
|-- Line_Name: string (nullable = true)
|-- Distance: integer (nullable = true)
|-- Number_of_Stops: integer (nullable = true)
|-- Departure_Hour: integer (nullable = true)
|-- Peak_Hour: integer (nullable = true)
|-- Day_Type: string (nullable = true)
|-- Weather: string (nullable = true)
|-- Traffic_Level: string (nullable = true)
|-- Delay_Minutes: integer (nullable = true)
|-- Average_Speed: integer (nullable = true)
|-- Fuel_Consumption: double (nullable = true)
|-- Efficiency_Class: string (nullable = true)
```

```
[8]: df.describe().show()
```

Comprehensive Route Performance Analysis - Q3 2023												
Summary		Operator	Service_Code	Line_Name		Distance	Number_of_Stops	Departure_Hour	Peak_Hour		Day_Type	Weather
Traffic_Level		Delay_Minutes	Average_Speed		Fuel_Consumption	Efficiency_Class						
	count	113936	113936	113936		113936	113936	113936	113936	113936	113936	1139
	mean	113936	113936	113936		113936	113936	113936	113936	113936	113936	1139
	LL	113936	113936	113936		113936	113936	113936	113936	113936	113936	1139
	stddev	113936	113936	113936		113936	113936	113936	113936	113936	113936	1139
	min	113936	113936	113936		113936	113936	113936	113936	113936	113936	1139
Detailed Performance Metrics - Route 101												
		mean	113936	113936	113936	113936	113936	113936	113936	113936	113936	1139
		stddev	113936	113936	113936	113936	113936	113936	113936	113936	113936	1139
		min	113936	113936	113936	113936	113936	113936	113936	113936	113936	1139
		max	113936	113936	113936	113936	113936	113936	113936	113936	113936	1139
		in	113936	113936	113936	113936	113936	113936	113936	113936	113936	1139
Summary of Key Findings												
The data shows a consistent performance across all metrics, with a slight increase in traffic levels during peak hours.												

```
[12]: from pyspark.ml.feature import VectorAssembler
```

```
feature_columns = [
    "Distance",
    "Number_of_Stops",
    "Departure_Hour",
    "Peak_Hour"
]

assembler = VectorAssembler(
    inputCols=feature_columns,
    outputCol="features"
)

df = assembler.transform(df)

df.select("features", "label").show(5, truncate=False)
```

```
+-----+-----+
|features|label|
+-----+-----+
|[437.0,264.0,7.0,1.0]|0.0|
|[437.0,109.0,7.0,1.0]|0.0|
|[721.0,3854.0,7.0,1.0]|2.0|
|[721.0,3854.0,7.0,1.0]|2.0|
|[721.0,3854.0,7.0,1.0]|2.0|
+-----+-----+
only showing top 5 rows
```

```
[15]: from pyspark.ml.classification import LogisticRegression
      from pyspark.ml.evaluation import MulticlassClassificationEvaluator

      # Train model
      lr = LogisticRegression(featuresCol="features", labelCol="label")

      lr_model = lr.fit(train_data)

      # Prediction
      lr_predictions = lr_model.transform(test_data)

      # Show predictions
      lr_predictions.select(
          "Efficiency_Class",
          "label",
          "prediction",
          "probability"
      ).show(10, truncate=False)
```

```
+-----+-----+-----+-----+
|Efficiency_Class|label|prediction|probability|
+-----+-----+-----+-----+
|Low            |0.0  |0.0       |[1.0,3.993244806516788E-40,7.8868129659135675E-199]|
|Low            |0.0  |0.0       |[1.0,3.993244806516788E-40,7.8868129659135675E-199]|
|Low            |0.0  |0.0       |[1.0,3.993244806516788E-40,7.8868129659135675E-199]|
|Low            |0.0  |0.0       |[1.0,3.993244806516788E-40,7.8868129659135675E-199]|
|Low            |0.0  |0.0       |[1.0,3.993244806516788E-40,7.8868129659135675E-199]|
|Low            |0.0  |0.0       |[1.0,3.993244806516788E-40,7.8868129659135675E-199]|
|Low            |0.0  |0.0       |[1.0,3.993244806516788E-40,7.8868129659135675E-199]|
|Low            |0.0  |0.0       |[1.0,3.993244806516788E-40,7.8868129659135675E-199]|
|Low            |0.0  |0.0       |[1.0,3.993244806516788E-40,7.8868129659135675E-199]|
|Low            |0.0  |0.0       |[1.0,3.993244806516788E-40,7.8868129659135675E-199]|
+-----+-----+-----+-----+
only showing top 10 rows
```

```
[19]: print("=" * 40)
      print("Model Comparison")
      print("=" * 40)

      print("Logistic Regression :", round(lr_accuracy, 4))
      print("Decision Tree       :", round(dt_accuracy, 4))
      print("Random Forest        :", round(rf_accuracy, 4))
```

```
=====
Model Comparison
=====
Logistic Regression : 1.0
Decision Tree       : 1.0
Random Forest       : 0.9958
```

```
[20]: rf_predictions.groupBy("label", "prediction").count().show()
```

```
+-----+-----+-----+
|label|prediction|count|
+-----+-----+-----+
| 1.0|      1.0| 7507|
| 0.0|      1.0|   95|
| 2.0|      2.0| 6399|
| 0.0|      0.0| 8600|
+-----+-----+-----+
```

[1]: `from pyspark.sql import SparkSession`

```
spark = SparkSession.builder \
    .appName("Predicting Bus Fuel Consumption for Operational Efficiency") \
    .getOrCreate()
```

[2]: `df = spark.read.csv(
 r"C:\Users\ASUS\Documents\Bus project\data\bus_fuel_consumption_114k.csv",
 header=True,
 inferSchema=True
)`
`df.show(5)`

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|Operator|Service_Code|Line_Name|Distance|Number_of_Stops|Departure_Hour|Peak_Hour|Day_Type|Weather|Traffic_Level|Delay_Minutes|Average_Speed|Fuel_Consumption|Efficiency_Class|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|Stagecoach South|PK0002571:21|46|437|264|7|1|Weekday|Rain|High|17|40|210.65|Low|
|Stagecoach South|PK0002571:21|46|437|264|7|1|Weekday|Rain|High|13|39|209.38|Medium|
|Stagecoach South|PK0002571:21|46|437|264|7|1|Weekday|Clear|High|21|36|195.43|Low|
|Stagecoach South|PK0002571:21|46|437|264|7|1|Weekday|Clear|Medium|5|53|168.42|Medium|
|Stagecoach South|PK0002571:21|46|437|264|7|1|Weekday|Rain|High|27|25|209.19|Low|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

[4]: `df.show(10, truncate=False)`

```
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|Operator|Service_Code|Line_Name|Distance|Number_of_Stops|Departure_Hour|Peak_Hour|Day_Type|Weather|Traffic_Level|Delay_Minutes|Average_Speed|Fuel_Consumption|Efficiency_Class|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|Stagecoach South|PK0002571:21|46|437|264|7|1|Weekday|Rain|High|17|40|210.65|Low|
|Stagecoach South|PK0002571:21|46|437|264|7|1|Weekday|Rain|High|13|39|209.38|Medium|
|Stagecoach South|PK0002571:21|46|437|264|7|1|Weekday|Clear|High|21|36|195.43|Low|
|Stagecoach South|PK0002571:21|46|437|264|7|1|Weekday|Clear|Medium|5|53|168.42|Medium|
|Stagecoach South|PK0002571:21|46|437|264|7|1|Weekday|Rain|High|27|25|209.19|Low|
|Stagecoach South|PK0002571:21|46|437|264|7|1|Weekend|Clear|Medium|5|52|64.9|Medium|
|Stagecoach South|PK0002571:21|46|437|264|7|1|Weekend|Clear|High|15|43|97.29|Medium|
|Stagecoach South|PK0002571:21|46|437|264|7|1|Weekday|Rain|High|14|43|11.2|Medium|
|Stagecoach South|PK0002571:21|46|437|264|7|1|Weekday|Clear|High|12|43|93.19|Medium|
|Stagecoach South|PK0002571:21|46|437|264|7|1|Weekday|Rain|High|18|39|10.75|Low|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
only showing top 10 rows
```

```
[7]: from pyspark.sql.functions import col, when, count

missing = df.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in df.columns
])

missing.show()

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|Operator|Service_Code|Line_Name|Distance|Number_of_Stops|Departure_Hour|Peak_Hour|Day_Type|Weather|Traffic_Level|Delay_Minutes|Average_Speed|Fuel_Consumption|Efficiency_Class|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|      0|          0|          0|          0|          0|          0|          0|          0|          0|          0|          0|          0|          0|          0|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

[8]: total_rows = df.count()
unique_rows = df.dropDuplicates().count()

print("Total Rows:", total_rows)
print("Unique Rows:", unique_rows)
print("Duplicate Rows:", total_rows - unique_rows)

Total Rows: 114000
Unique Rows: 113936
Duplicate Rows: 64

[9]: df.describe().show()

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|summary|Operator|Service_Code|Line_Name|Distance|Number_of_Stops|Departure_Hour|Peak_Hour|Day_Type|Weather|Traffic_Level|Delay_Minutes|Average_Speed|Fuel_Consumption|Efficiency_Class|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|count|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|
|mean|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|
|stddev|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|
|min|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|
|max|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|114000|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

[10]: df = df.dropDuplicates()

print("Rows after removing duplicates:", df.count())

Rows after removing duplicates: 113936

[11]: total_rows = df.count()
unique_rows = df.dropDuplicates().count()

print("Total Rows:", total_rows)
print("Unique Rows:", unique_rows)
print("Duplicate Rows:", total_rows - unique_rows)

Total Rows: 113936
Unique Rows: 113936
Duplicate Rows: 0
```

```
[13]: from pyspark.sql.functions import avg, round

df.groupBy("Weather") \
    .agg(
        round(avg("Fuel_Consumption"), 2).alias("Average_Fuel_Consumption")
    ) \
    .show()
```

```
+-----+-----+
|Weather|Average_Fuel_Consumption|
+-----+-----+
|    Fog|                412.34|
|   Clear|                366.48|
|    Rain|                406.02|
+-----+-----+
```

```
[14]: df.groupBy("Traffic_Level") \
    .agg(
        round(avg("Fuel_Consumption"), 2).alias("Average_Fuel_Consumption")
    ) \
    .show()
```

```
+-----+-----+
|Traffic_Level|Average_Fuel_Consumption|
+-----+-----+
|         High|                430.49|
|         Low|                306.52|
|        Medium|                351.84|
+-----+-----+
```

```
[19]: numeric_columns = [
    "Distance",
    "Number_of_Stops",
    "Delay_Minutes",
    "Average_Speed",
    "Fuel_Consumption"
]

for col_name in numeric_columns[:-1]:
    corr = df.stat.corr(col_name, "Fuel_Consumption")
    print(f"{col_name} vs Fuel_Consumption = {corr:.3f}")
```

```
Distance vs Fuel_Consumption = 0.991
Number_of_Stops vs Fuel_Consumption = -0.102
Delay_Minutes vs Fuel_Consumption = 0.058
Average_Speed vs Fuel_Consumption = -0.055
```

```
[20]: df.printSchema()

root
|-- Operator: string (nullable = true)
|-- Service_Code: string (nullable = true)
|-- Line_Name: string (nullable = true)
|-- Distance: integer (nullable = true)
|-- Number_of_Stops: integer (nullable = true)
|-- Departure_Hour: integer (nullable = true)
|-- Peak_Hour: integer (nullable = true)
|-- Day_Type: string (nullable = true)
|-- Weather: string (nullable = true)
|-- Traffic_Level: string (nullable = true)
|-- Delay_Minutes: integer (nullable = true)
|-- Average_Speed: integer (nullable = true)
|-- Fuel_Consumption: double (nullable = true)
|-- Efficiency_Class: string (nullable = true)
```

```
[23]: df.select(
      "Operator",
      "Operator_Index",
      "Weather",
      "Weather_Index",
      "Traffic_Level",
      "Traffic_Index",
      "Day_Type",
      "Day_Index"
    ).show(10, truncate=False)
```

Operator	Operator_Index	Weather	Weather_Index	Traffic_Level	Traffic_Index	Day_Type	Day_Index
Stagecoach South	0.0	Rain	1.0	High	0.0	Weekday	0.0
Stagecoach South	0.0	Fog	2.0	Low	2.0	Weekday	0.0
Stagecoach South	0.0	Clear	0.0	High	0.0	Weekday	0.0
Stagecoach South	0.0	Clear	0.0	Medium	1.0	Weekday	0.0
Stagecoach South	0.0	Rain	1.0	Medium	1.0	Weekday	0.0
Stagecoach South	0.0	Clear	0.0	Medium	1.0	Weekday	0.0
Stagecoach South	0.0	Fog	2.0	Low	2.0	Weekday	0.0
Stagecoach South	0.0	Clear	0.0	Medium	1.0	Weekend	1.0
Stagecoach South	0.0	Clear	0.0	Medium	1.0	Weekday	0.0
Stagecoach South	0.0	Clear	0.0	High	0.0	Weekend	1.0

only showing top 10 rows

```
[26]: df.select(
      "features",
      "Fuel_Consumption"
    ).show(5, truncate=False)
```

features	Fuel_Consumption
[437.0,264.0,7.0,1.0,19.0,35.0,0.0,1.0,0.0,0.0]	209.81
[437.0,109.0,7.0,1.0,8.0,50.0,0.0,2.0,2.0,0.0]	158.65
[721.0,3854.0,7.0,1.0,16.0,40.0,0.0,0.0,0.0,0.0]	393.67
[721.0,3854.0,7.0,1.0,7.0,46.0,0.0,0.0,1.0,0.0]	335.09
[721.0,3854.0,7.0,1.0,12.0,46.0,0.0,1.0,1.0,0.0]	359.41

only showing top 5 rows

```
[27]: train_data, test_data = df.randomSplit([0.8, 0.2], seed=42)
```

```
print("Training Rows :", train_data.count())
print("Testing Rows :", test_data.count())
```

```
Training Rows : 91335
Testing Rows : 22601
```

```
[30]: from pyspark.ml.evaluation import RegressionEvaluator
```

```
rmse_evaluator = RegressionEvaluator(
    labelCol="Fuel_Consumption",
    predictionCol="prediction",
    metricName="rmse"
)

mae_evaluator = RegressionEvaluator(
    labelCol="Fuel_Consumption",
    predictionCol="prediction",
    metricName="mae"
)

r2_evaluator = RegressionEvaluator(
    labelCol="Fuel_Consumption",
    predictionCol="prediction",
    metricName="r2"
)

lr_rmse = rmse_evaluator.evaluate(lr_predictions)
lr_mae = mae_evaluator.evaluate(lr_predictions)
lr_r2 = r2_evaluator.evaluate(lr_predictions)

print("Linear Regression Results")
print("RMSE =", lr_rmse)
print("MAE =", lr_mae)
print("R2 =", lr_r2)
```

```
Linear Regression Results
RMSE = 89.20656425133815
MAE = 30.762759798020173
R2 = 0.9884348432003315
```

```
[31]: from pyspark.ml.regression import DecisionTreeRegressor
```

```
dt = DecisionTreeRegressor(
    featuresCol="features",
    labelCol="Fuel_Consumption",
    maxDepth=8,
    seed=42
)

dt_model = dt.fit(train_data)
```

```
[32]: dt_predictions = dt_model.transform(test_data)
```

```
[33]: dt_rmse = rmse_evaluator.evaluate(dt_predictions)
dt_mae = mae_evaluator.evaluate(dt_predictions)
dt_r2 = r2_evaluator.evaluate(dt_predictions)

print("Decision Tree Results")
print("RMSE =", dt_rmse)
print("MAE =", dt_mae)
print("R2 =", dt_r2)
```

```
Decision Tree Results
RMSE = 48.810046072671916
MAE = 20.8890988610486
R2 = 0.9965376072127355
```



```
[36]: rf_rmse = rmse_evaluator.evaluate(rf_predictions)
      rf_mae = mae_evaluator.evaluate(rf_predictions)
      rf_r2 = r2_evaluator.evaluate(rf_predictions)
```

```
print("Random Forest Results")
print("RMSE =", rf_rmse)
print("MAE =", rf_mae)
print("R2 =", rf_r2)
```

```
Random Forest Results
RMSE = 46.33856395961204
MAE = 16.15014256357848
R2 = 0.9968793645278804
```

```
[37]: comparison = [
      ("Linear Regression", lr_rmse, lr_mae, lr_r2),
      ("Decision Tree", dt_rmse, dt_mae, dt_r2),
      ("Random Forest", rf_rmse, rf_mae, rf_r2)
    ]
```

```
comparison_df = spark.createDataFrame(
    comparison,
    ["Model", "RMSE", "MAE", "R2"]
)
```

```
comparison_df.show(truncate=False)
```

Model	RMSE	MAE	R2
Linear Regression	89.20656425133815	30.762759798020173	0.9884348432003315
Decision Tree	48.810046072671916	20.8890988610486	0.9965376072127355
Random Forest	46.33856395961204	16.15014256357848	0.9968793645278804

```
[42]: print("Dataset Shape")
      print(f"Rows: {df.count()}")
      print(f"Columns: {len(df.columns)}")
```

```
print("\nColumn Names")
print(df.columns)
```

```
print("\nSchema")
df.printSchema()
```

```
Dataset Shape
Rows: 113936
Columns: 19
```

```
Column Names
```

```
['Operator', 'Service_Code', 'Line_Name', 'Distance', 'Number_of_Stops', 'Departure_Hour', 'Peak_Hour', 'Day_Type', 'Weather', 'Traffic_Level', 'Delay_Minutes', 'Average_Speed', 'Fuel_Consumption', 'Efficiency_Class', 'Operator_Index', 'Weather_Index', 'Traffic_Index', 'Day_Index', 'features']
```

```
Schema
root
```

```
-- Operator: string (nullable = true)
-- Service_Code: string (nullable = true)
-- Line_Name: string (nullable = true)
-- Distance: integer (nullable = true)
-- Number_of_Stops: integer (nullable = true)
-- Departure_Hour: integer (nullable = true)
-- Peak_Hour: integer (nullable = true)
-- Day_Type: string (nullable = true)
-- Weather: string (nullable = true)
-- Traffic_Level: string (nullable = true)
-- Delay_Minutes: integer (nullable = true)
-- Average_Speed: integer (nullable = true)
-- Fuel_Consumption: double (nullable = true)
-- Efficiency_Class: string (nullable = true)
-- Operator_Index: double (nullable = false)
-- Weather_Index: double (nullable = false)
-- Traffic_Index: double (nullable = false)
```

```
[43]: df.describe().show(truncate=False)
```

	Operator	Service_Code	Line_Name	Distance	Number_of_Stops	Departure_Hour	Peak_Hour	Day_Type	Weather
count	113936	113936	113936	113936	113936	113936	113936	113936	113936
mean	NULL	NULL	151.1586777632317	859.3334064738099	1766.2936122033423	7.399461100968965	0.745664232551608	NULL	NULL
stddev	NULL	NULL	263.2209752633096	1963.4100800271256	2024.9510335243306	2.2308125358827087	0.4354890921508494	NULL	NULL
min	Stagecoach South	PH0005857:119	1	16	10	0	0	Weekday	Clear
max	Stagecoach South	PK0002571:99	WPR	16592	12471	17	1	Weekend	Rain

```
[44]: from pyspark.sql.functions import col, when, count

missing = df.select([
    count.when(col(c).isNull(), c).alias(c)
    for c in df.columns
])

missing.show()
```

Operator	Service_Code	Line_Name	Distance	Number_of_Stops	Departure_Hour	Peak_Hour	Day_Type	Weather	Traffic_Level	Delay_Minutes	Average_Speed	Fuel_Consumption	Efficiency_Class	Operator_Index	Weather_Index	Traffic_Index	Day_Index	features
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

7.4 Duplicate Records

```
[45]: duplicates = df.count() - df.dropDuplicates().count()

print("Duplicate Records:", duplicates)

Duplicate Records: 0
```